

BLUECREST UNIVERSITY COLLEGE

SCHOOL OF TECHNOLOGY

**AN ONLINE SMART SHOPPING AND SELF-CHECKOUT SYSTEM WITH EMAIL RECEIPT AND QR
VERIFICATION**

BY

CHRISTOFRED

XXXXXXXXXX

**A SYSTEM DOCUMENTATION REPORT SUBMITTED TO BLUECREST UNIVERSITY COLLEGE IN
PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF MASTER OF SCIENCE IN
INFORMATION TECHNOLOGY**

SEPTEMBER 2026

ABSTRACT

This documentation presents the design, implementation, and deployment preparation of StoreNav, an online smart shopping and self-checkout system for supermarket environments. The system enables customers to browse products, scan product barcodes, manage a live cart, pay through Paystack, receive an email receipt with a QR code, and present the QR code for verification by security personnel before leaving the store.

The current system removes the earlier virtual room concept and focuses on a practical online shop workflow. It includes role-based access for customers, administrators, and security staff; email OTP verification during account creation; GHS currency support; SweetAlert-based user feedback; product images; customer order history; and a Hostinger-ready deployment configuration.

STUDENT'S DECLARATION

I, Akporh Christofred Adjei, declare that this system documentation report is based on the StoreNav system developed for an online smart shopping and self-checkout workflow. The documentation reflects the implemented modules, system behavior, screenshots, database structure, and deployment requirements.

Signature: _____ Date: _____

SUPERVISOR'S DECLARATION

I hereby declare that the preparation and presentation of this work was supervised in accordance with the guidelines for research work and system documentation at BlueCrest University College.

Supervisor: _____ Signature: _____ Date: _____

ACKNOWLEDGEMENT

I am grateful to God for the strength and guidance throughout this work. I also acknowledge the support of my supervisor, lecturers, family, colleagues, and all individuals who contributed ideas, feedback, and encouragement during the development and documentation of the StoreNav system.

TABLE OF CONTENTS

Abstract

Chapter One: Introduction

Chapter Two: Literature Review

Chapter Three: Research Methodology and System Analysis

Chapter Four: System Design and Implementation

Chapter Five: Testing, Deployment, Conclusion, and Recommendations

References

Appendices

CHAPTER ONE

INTRODUCTION

1.0 Background of the Study

Retail shopping continues to evolve as supermarkets adopt digital tools for product display, checkout, inventory management, and customer engagement. In many supermarket settings, customers still move through aisles, identify products manually, wait at checkout points, and depend on staff for assistance. These activities can create delays, especially when customers need quick product discovery, real-time cart totals, and a faster payment process.

StoreNav responds to this problem by providing a web-based smart shopping system. Instead of emphasizing a virtual room, the system now focuses on an online shop and self-checkout workflow where users can browse available products, scan barcodes, add items to a cart, make payment, and receive a verifiable receipt. Security staff can scan the receipt QR code at the exit to confirm the products purchased.

1.1 Problem Statement

Traditional supermarket checkout processes can be slow and fragmented. Customers often discover product prices only after selecting items, while payment and verification occur at a separate stage. This separation can lead to long queues, limited spending visibility, and weak coordination between customer payment records and exit verification.

There is therefore a need for a system that allows customers to scan products, monitor cart totals in Ghana cedis, pay electronically, receive a secure receipt, and allow security personnel to verify the transaction using a QR code.

1.2 General Objective

The general objective of this project is to design and document an online smart shopping and self-checkout system that supports customer scanning, cart management, Paystack payment, email receipt delivery, and QR-based receipt verification.

1.3 Specific Objectives

- Develop a customer-facing online shop with product browsing, search, filtering, and random product display.
- Provide barcode scanning and direct product addition to a live cart.
- Integrate Paystack payment using GHS as the system currency.
- Send email receipts after successful payment and include a QR code for security verification.
- Provide a security portal for receipt lookup, QR scanning, and verification.
- Provide an admin panel for products, categories, orders, inventory, analytics, customers, and settings.

- Prepare the system for Hostinger deployment using production-safe environment settings and database seeding.

1.4 Research Questions

1. How can supermarket customers scan and pay for products without relying on a cashier at every transaction point?
2. How can the system verify that a customer leaving the store has paid for the products purchased?
3. How can administrators manage product, order, inventory, and customer data from a central panel?
4. How can account verification, role-based access, and deployment settings improve readiness for hosting?

1.5 Scope of the Study

The scope covers a Laravel-based web application with customer, administrator, and security staff roles. It includes product browsing, barcode scanning, cart management, checkout, Paystack payment initialization and verification, email OTP account verification, receipt email generation, QR receipt verification, order history, product image support, and deployment preparation. The removed virtual room is outside the current system scope.

1.6 Significance of the Study

The system demonstrates how supermarkets can reduce checkout friction and strengthen payment verification through a web-based self-checkout process. It also provides a practical academic artefact that combines authentication, payment integration, email delivery, QR verification, database design, and role-based administration.

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

This chapter reviews concepts relevant to the StoreNav system, including smart retail systems, barcode-based product identification, online carts, digital payment, receipt verification, and role-based administration.

2.1 Smart Retail and Self-Checkout Systems

Smart retail systems use digital technologies to improve how customers discover products, make decisions, and complete purchases. Self-checkout systems shift part of the transaction process to customers, allowing them to scan products and complete payment with less dependence on cashier-operated counters.

2.2 Barcode Scanning and Product Identification

Barcode scanning provides a practical method for identifying products. In StoreNav, a barcode is associated with each product record. When the user scans a barcode, the system searches for the product, adds it to the user's cart, recalculates totals, and updates the cart badge.

2.3 Digital Payment Integration

Digital payment integration is important for self-checkout because payment must be verified before the transaction is considered complete. StoreNav uses Paystack to initialize payment and verify the callback reference. Successful payment changes the order status to paid, clears the active cart, and starts receipt delivery.

2.4 Email Receipts and QR Verification

A self-checkout process requires a reliable post-payment proof of purchase. StoreNav generates a unique receipt token for each paid order and includes a QR code in the email receipt. Security staff can scan the QR code or enter the token manually to confirm the order and mark it as verified.

2.5 Role-Based Access Control

Role-based access control separates system responsibilities. Customers can shop and view their own history. Administrators manage business data. Security staff verify paid receipts. Unauthorized users are routed to the login page so they can sign in or create an account instead of seeing generic unauthorized screens.

CHAPTER THREE

RESEARCH METHODOLOGY

3.0 Introduction

This chapter presents the research methodology used for the development, implementation, and documentation of the StoreNav online smart shopping and self-checkout system. The chapter explains the research design, research approach, study population, sampling technique, data sources, research instruments, system development method, system analysis procedure, testing approach, data analysis method, validity, reliability, and ethical considerations.

The methodology is structured to show how the shopping problem was studied and how the software artefact was developed as a response to that problem. Since the project is both an academic research work and a practical system implementation, the methodology combines research activities with software engineering activities. This makes it possible to connect the identified supermarket checkout problem to the actual modules implemented in the StoreNav system.

The chapter also explains how system requirements were translated into features such as account registration, email OTP verification, product browsing, barcode scanning, cart management, Paystack checkout, receipt email delivery, QR receipt verification, customer order history, administrative management, and Hostinger deployment preparation.

3.1 Research Design

The study adopted Design Science Research as the main research design. Design Science Research is appropriate for information technology projects because it emphasizes the creation of a useful artefact that solves an identified problem. The artefact is then evaluated to determine whether it performs the functions for which it was designed.

In this project, the artefact is the StoreNav system. The system was created to address the inconvenience of manual product selection, fragmented cart monitoring, delayed checkout, and weak exit verification in supermarket shopping. The design focuses on enabling customers to scan and pay for products themselves while administrators manage product and transaction data and security staff verify purchase receipts.

The research design followed five major stages: problem identification, requirement analysis, artefact design, system implementation, and evaluation. Each stage contributed to the final system. Problem identification focused on shopping and checkout delays. Requirement analysis converted the problem into functional needs. Artefact design defined the system modules and database structure. Implementation produced the Laravel application. Evaluation confirmed system behavior through tests and screenshots.

Research Design Stage	Application in StoreNav
Problem identification	The study identified manual shopping, checkout delays, limited cart visibility, and receipt verification as key supermarket process issues.

Requirement analysis	The project defined customer, administrator, and security requirements and translated them into system features.
Artefact design	The system architecture, database tables, routes, controllers, views, services, and role boundaries were planned.
Implementation	The StoreNav Laravel application was developed with product browsing, scanning, payment, receipt email, QR verification, and admin management.
Evaluation	Automated tests, browser screenshots, and document review were used to confirm that the implemented system matched the objectives.

3.2 Research Approach

The research used a pragmatic approach. A pragmatic approach was selected because the main objective was not only to explain a problem but also to produce a working system that can solve the problem in a practical supermarket environment. The approach allowed the study to combine user-centered analysis, software design, implementation, and testing.

The qualitative aspect of the approach was used to understand the shopping journey, the points where customers experience inconvenience, and the responsibilities of administrators and security personnel. This helped define the system boundaries and user roles. The technical aspect was used to build and test the software modules that support self-checkout and receipt verification.

The approach therefore links the academic investigation to the implemented system. For example, the problem of checkout delay is addressed by Paystack self-checkout, while the problem of proof of payment at the exit is addressed by email receipts and QR verification.

3.3 Population of the Study

The population of the study consists of supermarket customers, supermarket administrators or managers, and security personnel. These three groups were selected because they represent the main stakeholders in the StoreNav self-checkout process. The customer initiates the shopping activity, the administrator maintains the store data, and the security officer validates the transaction before exit.

Customers are central to the study because the system is designed to improve their shopping experience. They interact with the shop, scanner, cart, checkout, email receipt, and history modules. Supermarket administrators are included because the system requires accurate product records, product images, prices, barcodes, categories, inventory levels, and order monitoring. Security personnel are included because the system introduces QR-based receipt verification as a replacement for manual receipt inspection.

Population Group	Relevance to the Study
Customers	They represent the users who browse products, scan items, manage carts, make payments, and receive receipts.
Supermarket administrators	They represent the users responsible for product records, stock levels, orders, analytics, and general system configuration.
Security personnel	They represent the users responsible for confirming that paid products match a verified receipt before customers leave the store.

3.4 Sampling Technique and Sample Size

The study used purposive sampling because participants had to be selected based on their relevance to the supermarket shopping and checkout workflow. Randomly selecting users without considering their relationship to the system would not provide useful feedback on customer shopping, administrator management, and security verification tasks.

Purposive sampling makes it possible to select people who understand the process being studied. For example, a customer can evaluate whether the shopping and cart interfaces are usable; an administrator can evaluate whether product and order management are sufficient; and a security staff member can evaluate whether QR verification is practical at the exit point.

For prototype evaluation, the study can use a focused sample consisting of customers, administrators, and security personnel. The sample size can be adjusted based on institutional guidance, access to respondents, and the time available for testing. The important requirement is that each major system role is represented during evaluation.

Sample Category	Suggested Participants	Reason for Inclusion
Customers	10 to 20 users	To test registration, email OTP, product search, scanning, cart totals, checkout, and history.
Administrators	2 to 5 users	To test product management, order viewing, customer records, inventory, analytics, and settings.
Security staff	2 to 5 users	To test QR scanning, manual receipt lookup, receipt viewing, and verification marking.

3.5 Data Sources

The study used both primary and secondary data sources. Primary data refers to information gathered directly from observation, users, and system testing. Secondary data refers to information obtained from existing academic materials, technical documentation, payment integration references, and software development resources.

Primary data is important because it connects the project to real shopping activities. Observing the shopping process helps identify where delays occur and where automation can improve the workflow. User feedback helps determine whether the interface is easy to use and whether the self-checkout process is understandable. System testing provides evidence that the implemented modules function correctly.

Secondary data supports the theoretical and technical foundation of the system. Literature on smart retail, barcode scanning, online carts, electronic payment, and QR verification helps justify the design decisions used in StoreNav. Technical documentation supports the correct use of Laravel, Paystack, email delivery, and Hostinger deployment practices.

Data Source	Description
Primary data	Information gathered from users, observations, testing feedback, and interaction with the StoreNav prototype.
Secondary data	Information gathered from academic literature, online technical documentation,

	payment integration references, and related smart retail studies.
--	---

3.6 Research Instruments

The instruments used for the study include observation, questionnaires or informal interview questions, a system testing checklist, and screenshot-based documentation. Each instrument served a specific purpose in understanding the problem, confirming user needs, or validating the final system.

Observation was used to examine how customers usually browse products, identify prices, carry selected items, and complete payment. Questionnaires or informal interviews can be used to collect perceptions from customers, administrators, and security staff. The testing checklist was used to confirm whether each system function produced the expected result. Screenshots from the running system were used to document the implemented interfaces.

- Observation was used to understand how customers normally browse, select, and pay for products.
- Questionnaires or informal interviews can be used to gather feedback from customers, administrators, and security staff.
- A testing checklist was used to verify registration, OTP, login, scanning, cart, checkout, receipt email, QR verification, and admin management.
- Screenshots from the running system were used as evidence of implementation and to support system documentation.

Instrument	Information Collected	Contribution to the Study
Observation	Customer movement, checkout delays, staff dependence, product identification behavior.	Helped identify practical shopping problems.
Questionnaire/interview	User expectations, usability concerns, role-specific feedback.	Helped validate user requirements.
Testing checklist	Pass/fail status of system features.	Helped evaluate system functionality.
Screenshots	Visual evidence of implemented screens.	Supported system documentation and presentation.

3.7 System Development Methodology

The system was developed using an iterative prototyping methodology. Prototyping was selected because the system required visible user interfaces and practical workflow testing. It allowed the system to be reviewed and improved as requirements became clearer.

The initial direction of the project included a virtual room concept. During refinement, the system was redirected toward the more practical requirement of an online shop where users scan and pay for products themselves. The virtual room was therefore removed from the active system, and attention was placed on product browsing, scanner-based cart addition, Paystack checkout, email receipt delivery, and QR verification.

The iterative methodology allowed interface issues and functional issues to be addressed progressively. Product card sizes were adjusted, the product layout was changed to a regular grid, generic alerts were

replaced with SweetAlert, unauthorized actions were redirected to login, cart images were corrected, GHS currency was applied, and logout access was added for all roles.

5. Requirement analysis: the major system needs were identified, including self-checkout, payment, receipts, QR verification, admin management, and hosting readiness.
6. System design: the customer, admin, and security modules were separated and mapped to Laravel routes, controllers, services, views, and database tables.
7. Interface redesign: the system was redesigned using the selected template colors and theme, while removing the virtual room from the active workflow.
8. Implementation: the product shop, scanner, cart, checkout, Paystack integration, email OTP, order receipt, QR verification, order history, and admin panel were implemented.
9. Testing and refinement: automated tests and manual browser checks were used to verify the main workflows and correct discovered issues.
10. Deployment preparation: production environment settings, database seeding, and Hostinger hosting guidance were prepared.

3.8 System Analysis

System analysis was carried out by identifying the users, processes, inputs, outputs, data stores, and controls required by the StoreNav application. The analysis focused on the complete self-checkout cycle: account creation, email verification, product browsing, scanning, cart update, payment, receipt generation, and security verification.

The analysis showed that the system required seven major functional areas: customer shopping, authentication, cart and checkout, payment verification, receipt email, security verification, and administration. Each functional area was mapped to Laravel controllers, Blade views, service classes, database models, and routes.

3.8.1 System Users

User Role	Main Responsibilities
Customer	Create account, verify email OTP, browse products, scan products, manage cart, checkout, view order history, receive email receipt.
Administrator	Manage products and images, categories, customers, orders, inventory, analytics, settings, and security access links.
Security Staff	Scan receipt QR codes, search receipt tokens manually, inspect paid order details, and mark receipts as verified.

3.8.2 Functional Requirements

- The system shall allow users to create accounts and verify email ownership with a one-time password.
- The system shall allow customers to browse and search products in GHS.
- The system shall allow barcode scanning and cart updates through secure API routes.
- The system shall initialize Paystack payment and verify payment callbacks.

- The system shall send an email receipt containing order items, total amount, and a QR verification code.
- The system shall provide a security portal for receipt verification.
- The system shall provide an admin dashboard for product, inventory, order, analytics, and customer management.

3.8.3 Input Requirements

Input	Source	Purpose
Customer registration details	Registration form	Creates a customer account and starts email OTP verification.
Email OTP code	Customer email and OTP form	Confirms ownership of the registered email address.
Search/filter text	Shop page	Finds products by name, barcode, category, or price range.
Barcode value	Scanner or manual product barcode	Identifies the product to add to the cart.
Payment reference	Paystack callback	Verifies the completed payment transaction.
Receipt token or QR code	Email receipt/security portal	Finds and validates a paid order.
Product data	Admin product forms	Maintains product catalog, image, price, barcode, stock, and category.

3.8.4 Output Requirements

- The shop must display active products with images, category labels, GHS prices, and add-to-cart actions.
- The cart must display selected products, product images, quantities, subtotals, tax, and total amount.
- The checkout page must display the order total and payment provider options.
- The email receipt must display purchased items, total amount, and a QR code for security verification.
- The security portal must display receipt validity, customer details, products purchased, payment status, and verification status.
- The admin dashboard must display sales, orders, active users, conversion rate, charts, top products, and recent orders.

3.8.5 Process Requirements

11. The system validates the user's identity and role before protected actions are allowed.
12. The system verifies the customer's email address using a one-time password before continuing the customer workflow.
13. The system loads active products and randomizes the product list to prevent the same product order on every shop visit.
14. The system adds scanned products to the active cart and recalculates totals immediately.
15. The system creates an order and payment record before payment redirection.
16. The system verifies Paystack callback data before marking payment as successful.

17. The system generates a unique receipt token and QR code after successful payment.
18. The system records receipt verification when security staff confirm the order.

3.8.6 Data Storage Requirements

The system requires persistent storage for users, categories, products, carts, cart items, orders, order items, payments, inventory logs, email OTPs, sessions, cache records, and jobs. These records are managed through Laravel migrations and Eloquent models. The data structure ensures that temporary cart activity is separated from final order records.

The separation between cart items and order items is important because a cart can change before checkout, while an order must preserve the final purchased products. The payment table preserves provider references and payloads, while receipt verification fields on the order table record whether a customer has already been cleared by security.

3.8.7 Non-Functional Requirements

- Usability: alerts and confirmations use SweetAlert instead of generic browser alerts.
- Security: protected actions require authentication and role checks.
- Reliability: payment verification depends on Paystack reference validation before fulfillment.
- Maintainability: seeders are idempotent and configurable for production credentials.
- Deployability: the application includes Hostinger production environment guidance.

3.8.8 System Constraints

- The system depends on correct Paystack environment keys for live payment processing.
- The system depends on a working SMTP provider for OTP and receipt delivery.
- Camera-based barcode or QR scanning depends on browser permission and device camera availability.
- Product images depend on correct public storage configuration and the Laravel storage link.
- Shared hosting deployment requires the domain document root to point to the Laravel public directory.
- Background queue processing should run synchronously on shared hosting unless a worker is configured.

3.9 Development Tools and Technologies

Item	Description
Backend framework	Laravel 11 with controllers, Eloquent models, migrations, services, middleware, mailables, events, and jobs.
Frontend	Blade templates, Tailwind/Vite assets, JavaScript cart/scan interactions, SweetAlert dialogs, and QR scanner integration.
Database	MySQL or SQLite locally, with migrations for users, products, carts, orders, payments, inventory logs, sessions, jobs, and OTP records.
Payment	Paystack initialization and verification using configured public/secret keys and GHS currency.

Email	Laravel Mail for OTP verification and order receipt delivery.
Hosting target	Hostinger shared hosting with the domain pointed to the Laravel public directory.

3.10 Use Case Summary

Use Case	Actor	Outcome
Register account	Customer	A new account is created and routed to OTP verification.
Verify email OTP	Customer	The account becomes verified and the user can access protected shopping actions.
Scan product	Customer	The product is added to the active cart and totals are recalculated.
Checkout	Customer	An order and payment record are created and the user is sent to Paystack.
Verify payment	System	The Paystack callback validates the transaction and fulfills the order.
Verify receipt	Security staff	The QR/token is checked and marked as verified.
Manage products	Administrator	Products, images, prices, categories, stock, and active state are maintained.

3.11 Detailed Workflow Analysis

3.11.1 Customer Shopping Workflow

19. The customer opens the StoreNav shop or home page.
20. The customer searches or filters products by name, barcode, category, or price range.
21. The customer adds products directly or opens the scanner page to scan barcodes.
22. The cart API stores selected items against the authenticated customer's active cart.
23. The cart total is recalculated in GHS and displayed in the cart and checkout views.
24. The customer checks out and proceeds to Paystack payment.
25. After successful payment, the customer receives an email receipt and can view the order in history.

3.11.2 Administrator Workflow

26. The administrator logs in with an admin account.
27. The system redirects the administrator to the admin dashboard.
28. The administrator monitors revenue, order count, active users, conversion rate, top products, and recent orders.
29. The administrator manages products, product images, categories, stock, customers, orders, inventory logs, analytics, and settings.
30. The administrator can review order records and access security-related pages where necessary.

3.11.3 Security Verification Workflow

31. The security staff logs in with a staff account.
32. The system redirects the staff user to the security portal.

33. The staff member scans the QR code from the customer's email receipt or enters the receipt token manually.
34. The system loads the matching order and displays customer, product, payment, and verification details.
35. The staff member verifies the receipt if the transaction is valid and unverified.
36. The system records the verification timestamp and the user who performed the verification.

3.12 Data Analysis Method

The data analysis method for this project is descriptive. User requirements, observed shopping challenges, and system testing results are described and interpreted in relation to the objectives of the study. Functional test results are used to determine whether the implemented system satisfies the required customer, admin, and security workflows.

Descriptive analysis is suitable because the study evaluates whether the artefact performs required operations rather than testing a statistical hypothesis. The analysis therefore focuses on whether each objective is represented by a working module and whether each user role can complete its required tasks.

Objective	Evidence Used for Analysis
Develop customer self-checkout	Shop, scan, cart, checkout, and Paystack workflow screenshots and tests.
Support receipt verification	Receipt token, QR email generation, security portal, and verification tests.
Support administration	Admin dashboard, products, orders, inventory, analytics, and customer management screens.
Prepare for hosting	Hostinger environment template, production seeder, build output, and deployment checklist.

3.13 Testing Methodology

Testing was carried out using both automated and manual methods. Automated tests were used to confirm core backend behavior such as cart updates, checkout, receipt verification, order history, and email OTP verification. Manual browser testing was used to inspect the user interface, screenshots, navigation, role redirects, SweetAlert feedback, and page layout.

The testing methodology covered unit tests, feature tests, role-based manual testing, and visual screenshot review. Unit tests confirmed smaller pieces of business logic. Feature tests confirmed complete request/response behavior. Manual testing confirmed that the browser interface behaved as expected and that screenshots accurately represented the final system.

Testing Type	Area Covered	Expected Result
Unit testing	Cart service and checkout logic.	Products are added correctly and order totals are calculated.
Feature testing	API cart, checkout, security verification, customer history, and OTP verification.	Protected workflows return expected responses and update database records.
Manual browser testing	Customer, admin, and security screens.	Pages render correctly and role redirects work.

Build testing	Vite production assets.	Production CSS and JavaScript are generated successfully.
Document render testing	Word export and screenshot placement.	Figures, captions, headings, and tables render cleanly.

3.14 Validity and Reliability

Validity refers to whether the study measures and documents the intended system behavior. In this project, validity was improved by basing the documentation on the actual implemented system rather than a theoretical design only. Screenshots were captured from the running application and the documentation was aligned with existing Laravel routes, controllers, services, migrations, and views.

Reliability refers to whether the system and documentation process can produce consistent results when repeated. Reliability was supported through automated tests, repeatable database seeders, production environment templates, and a reusable document builder script. These tools make it possible to regenerate the documentation and verify the system again after changes.

3.15 Ethical Considerations

The system was developed with consideration for user privacy, payment safety, and access control. Test accounts and sample data were used during documentation. Production credentials such as Paystack keys, email passwords, and database passwords should be stored only in the server environment file and should not be exposed in public documentation or source repositories.

The system also respects role boundaries. Customers should only see their own history and cart data. Administrators should manage operational data, and security staff should only verify receipts. Email OTP verification helps reduce unauthorized account usage by confirming ownership of the email address used during registration.

- No real customer payment record should be used in documentation without permission.
- Live Paystack secret keys and Gmail app passwords must not be printed in reports.
- Test data should be clearly separated from production data.
- The security portal should be accessible only to authenticated staff or administrators.
- Receipt verification should record who verified the order and when verification occurred.

3.16 Summary

This chapter explained the methodology used to develop and document the StoreNav system. It described the Design Science Research approach, the target population, sampling, data sources, instruments, development process, system analysis, testing approach, and ethical issues. The next chapter presents the system design and implementation with screenshots from the running application.

CHAPTER FOUR

SYSTEM DESIGN AND IMPLEMENTATION

4.0 System Overview

This chapter presents the detailed design and implementation of the StoreNav online smart shopping and self-checkout system. It explains the system architecture, user interfaces, customer module, authentication module, cart and checkout module, Paystack payment module, email receipt module, QR verification module, administrator module, database design, route structure, security design, deployment design, and implementation decisions.

StoreNav is structured around three major interfaces: the customer shop, the administrator panel, and the security verification portal. The customer interface supports product discovery, barcode scanning, cart management, payment, receipt delivery, and order history. The administrator interface manages products, images, prices, categories, orders, customers, inventory, analytics, and settings. The security portal verifies receipt QR codes and confirms that paid orders are valid before the customer exits the supermarket.

The current implementation removes the earlier virtual room from the active user experience. The redesigned system is now centered on a practical self-checkout journey in which the customer can scan products, pay independently, and receive a verifiable email receipt.

4.0.1 System Architecture

The system follows a layered Laravel architecture. The presentation layer is built with Blade templates, Tailwind/Vite assets, JavaScript interactions, SweetAlert dialogs, and QR scanner scripts. The application layer is handled by controllers and services. The data layer is implemented through Eloquent models and database migrations. External services include Paystack for payment processing and SMTP mail for OTP and receipt delivery.

Layer	Implementation	Responsibility
Presentation layer	Blade views, layouts, JavaScript, Tailwind/Vite assets.	Displays the customer shop, scan page, cart, checkout, history, admin panel, and security portal.
Controller layer	CustomerController, WebAuthController, API controllers, Admin controllers, SecurityPortalController.	Receives requests, validates inputs, selects services, and returns views or JSON responses.
Service layer	CartService, PaystackService, OrderFulfillmentService, RecommendationService.	Handles business rules such as cart calculation, payment setup, order fulfillment, receipt generation, and recommendations.
Data layer	Eloquent models and migrations.	Stores users, products, carts, orders, payments, OTPs, inventory logs, sessions, cache, and jobs.
External services	Paystack and SMTP mail provider.	Processes payments and sends OTP or receipt emails.

4.0.2 High-Level Data Flow

37. A guest user opens the shop and can browse products but protected actions route the user to login.
38. A customer creates an account and verifies the email address using OTP.
39. The customer browses products or scans a barcode to add a product to the active cart.
40. The cart service updates quantities, totals, and product payloads, including product images.
41. The checkout controller converts the cart into an order and initializes payment through Paystack.
42. Paystack redirects back to the application with a transaction reference.
43. The payment callback verifies the reference and confirms that the transaction matches the order amount.
44. The fulfillment service marks the order paid, clears the cart, updates fulfillment fields, and sends the receipt email.
45. The receipt email includes the purchased items and an embedded QR code that points to the security verification route.
46. Security staff scan or enter the receipt token and mark the receipt as verified after checking the order.

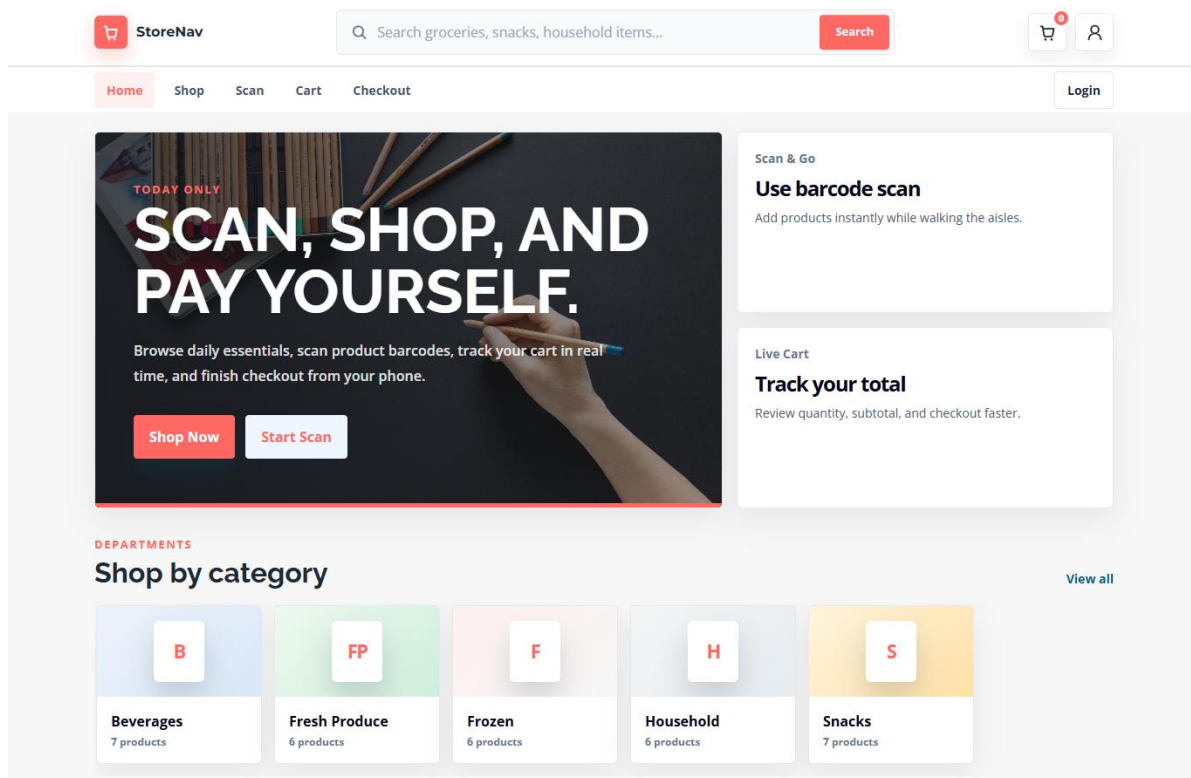


Figure 4.1: Customer home screen showing the online shop focus.

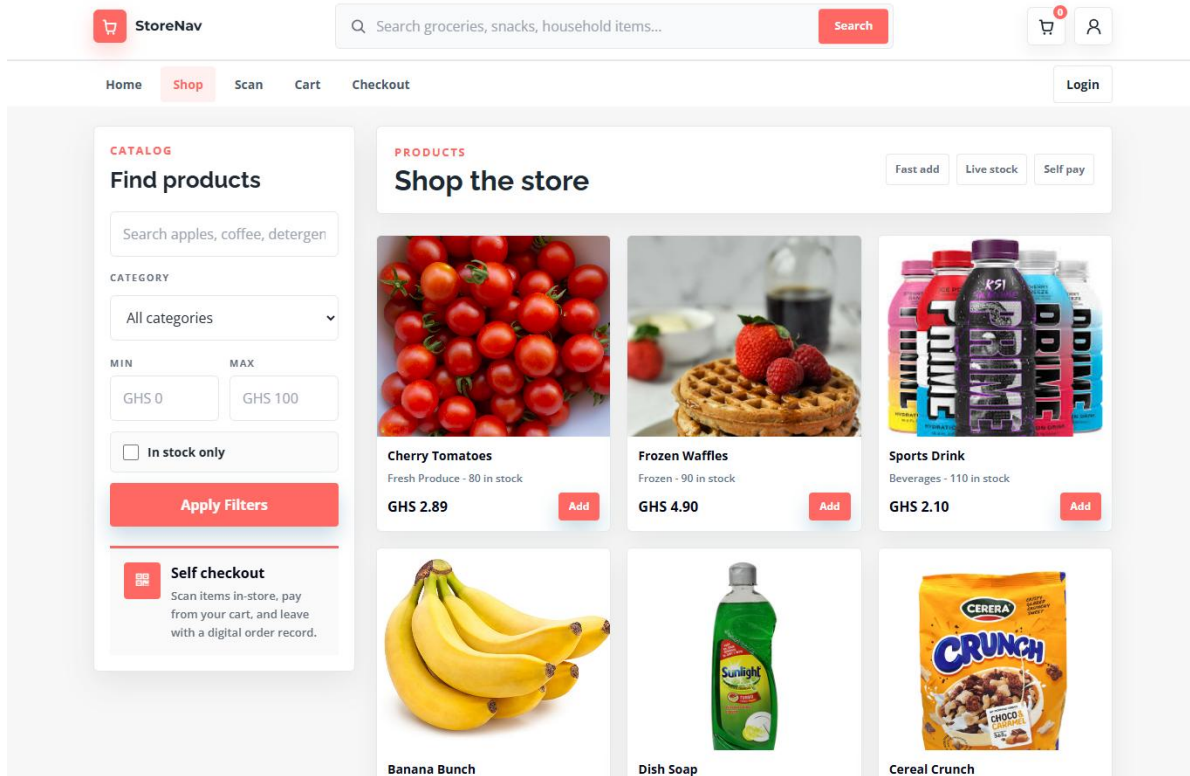


Figure 4.2: Product listing page with search, filters, random display, images, and GHS pricing.

4.1 Customer Module

The customer module is the main shopping interface. It contains the home page, shop page, product details page, scan page, cart page, checkout page, and order history page. The module was redesigned to make the first screen immediately useful as a shopping system rather than a landing page or virtual navigation room. The customer sees products, categories, cart status, search controls, and direct shopping actions.

The customer module uses a top navigation bar on desktop and a bottom tab bar on mobile. The navigation includes home, shop, scan, cart, checkout, and history where appropriate. The login action appears on the right side of the desktop navbar for guests, while authenticated users get logout access. The design uses GHS pricing throughout the customer journey.

4.1.1 Home Page Implementation

The home page is loaded by CustomerController. It retrieves active products, trending products, and product categories. Product cards show images where available, category context, prices in GHS, and add-to-cart actions. The home page also acts as a starting point for customers who want to move directly to scanning or browsing.

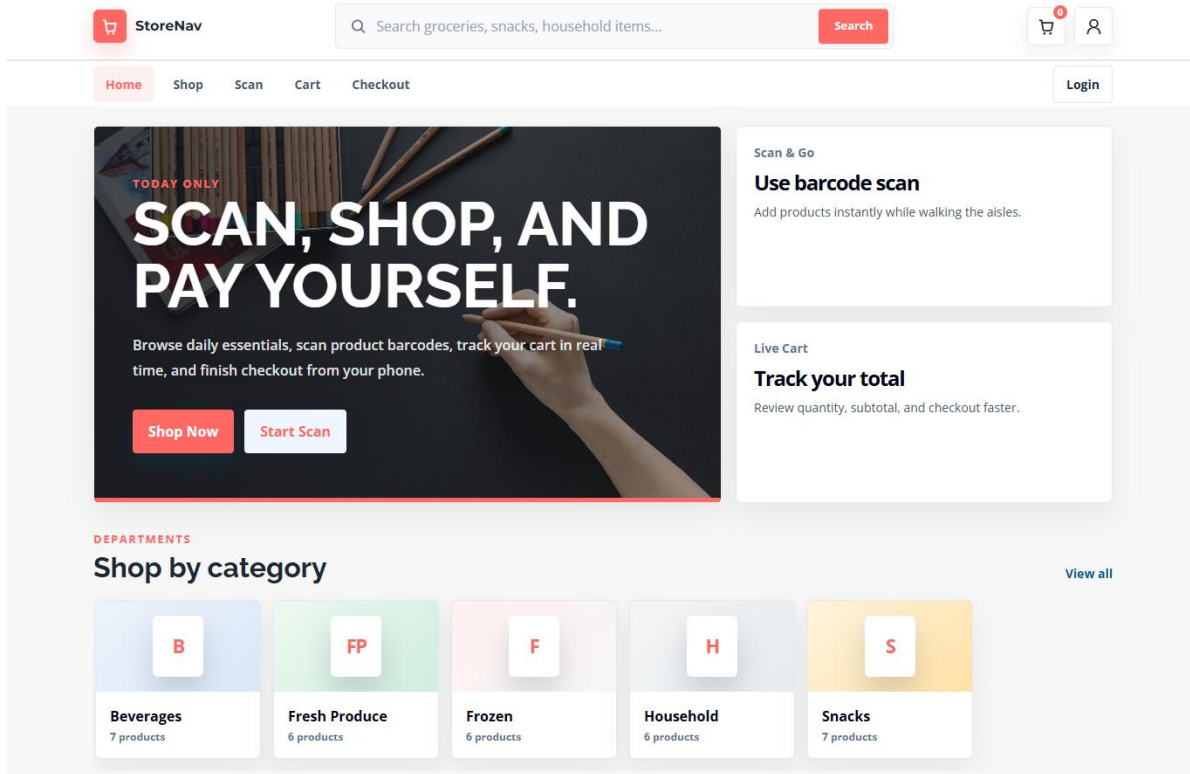


Figure 4.3: Home page showing featured shopping actions, categories, and product cards.

4.1.2 Shop Page Implementation

The shop page provides product browsing with search, category filtering, price range filtering, and randomized product display. Randomization prevents the same product order from appearing every time the shop is opened. Product cards were reduced in size to avoid oversized layouts and arranged in a regular grid for a cleaner shopping experience.

The product listing is served through the API ProductController. The controller filters active products and applies `inRandomOrder` before pagination. This keeps the shop visually fresh while preserving category and search functionality.

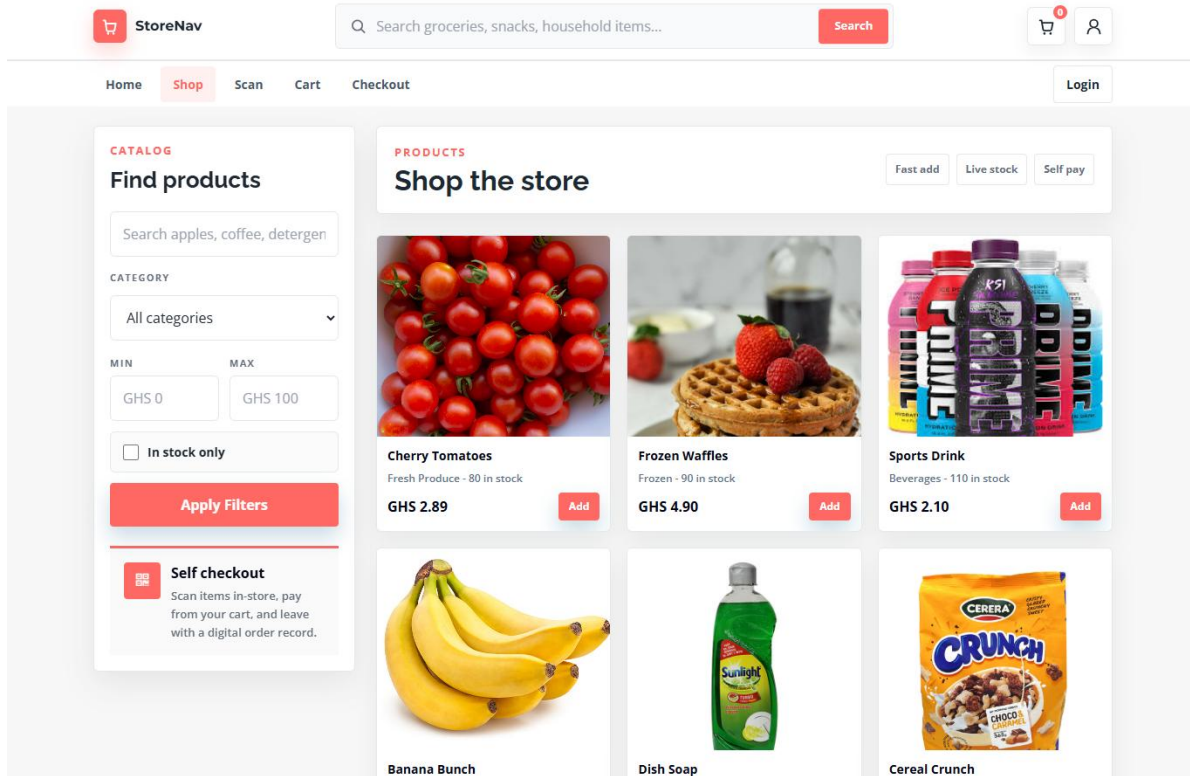


Figure 4.4: Shop page showing product search, category filters, and regular product grid.

4.1.3 Product Image Handling

Product images are stored using Laravel's public storage disk. The admin product form uploads images to the products directory under public storage. Customer shop, product, cart, checkout, and admin screens read the product image path and display the image through the storage URL. This ensures that the same uploaded image appears across the entire system.

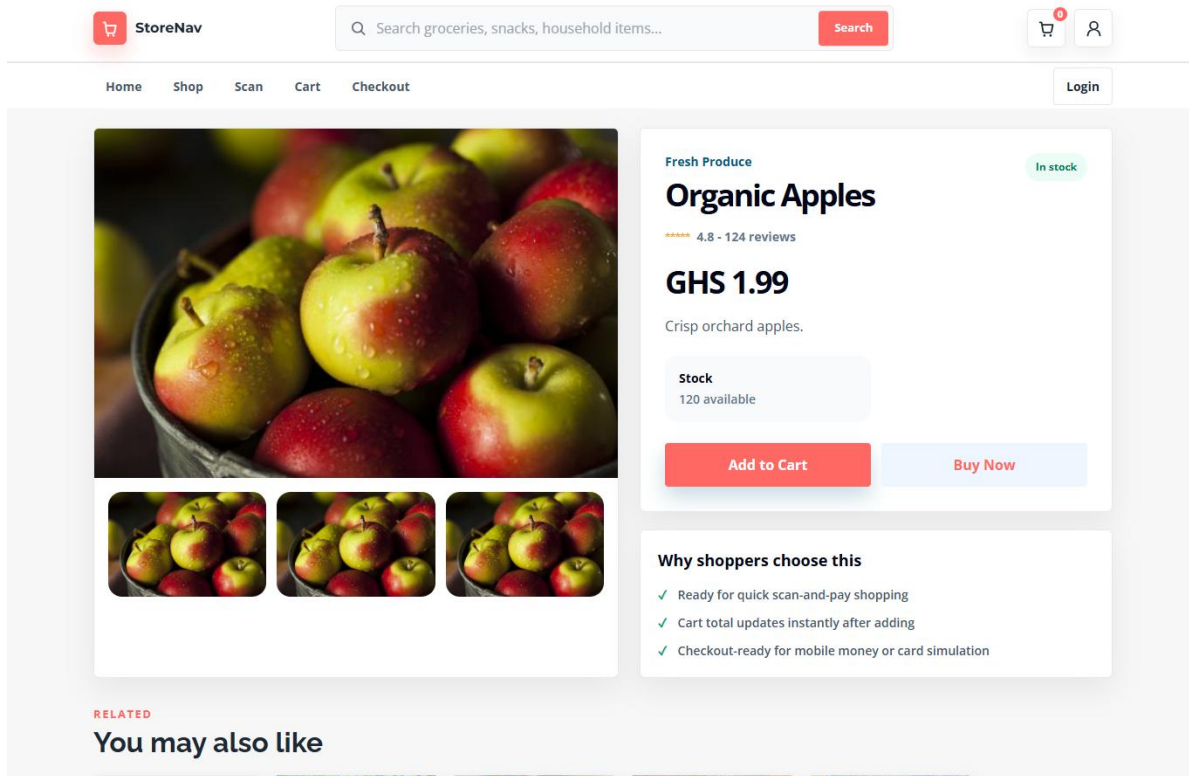


Figure 4.5: Product detail page showing uploaded product imagery and product information.

4.1.4 Cart Page Implementation

The cart page displays selected products, quantities, product images, item totals, subtotal, tax, and final total. The cart page communicates with protected API routes, which means unauthenticated users are routed to login before they can modify protected cart data. This behavior prevents unauthorized cart manipulation and aligns with the account-based order history requirement.

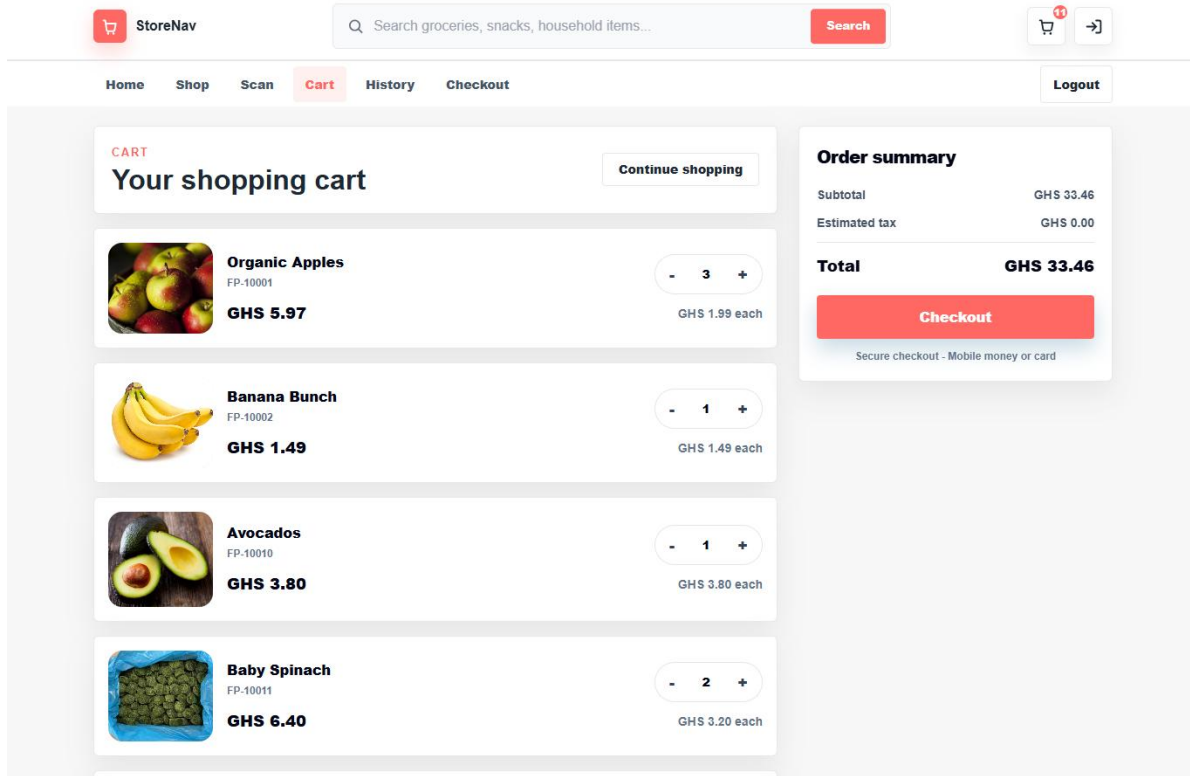


Figure 4.6: Cart page showing product images, quantities, item totals, and checkout summary.

4.1.5 Checkout Page Implementation

The checkout page summarizes the cart and provides the payment action. It supports Paystack as the main electronic payment provider. When checkout begins, the API CheckoutController creates the order and payment records, then calls PaystackService to initialize payment. If Paystack returns an authorization URL, the browser redirects the customer to complete payment.

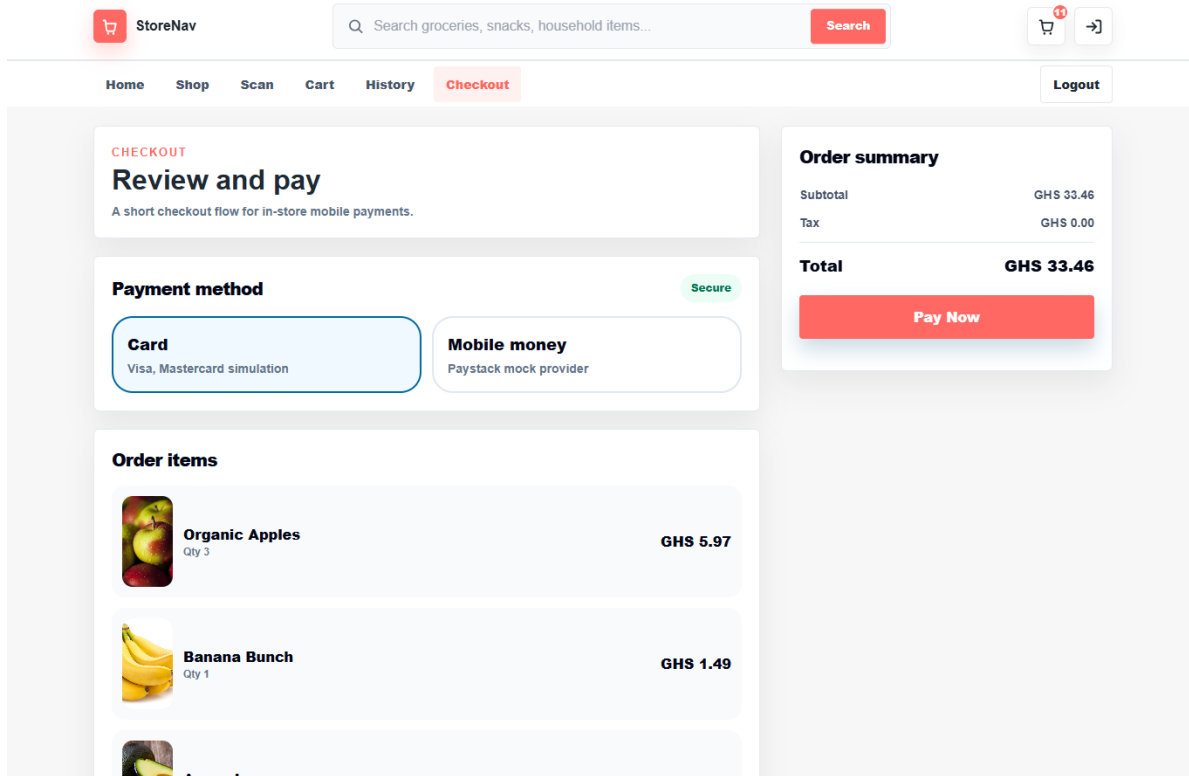


Figure 4.7: Checkout page showing payment method selection, order items, and GHS total.

4.1.6 Order History Implementation

The order history page allows authenticated customers to view their previous purchases. It loads the customer's orders with related order items, products, and payment records. Each history entry shows the order status, payment status, purchased products, totals, and whether a receipt token exists. This gives customers a record of completed transactions.

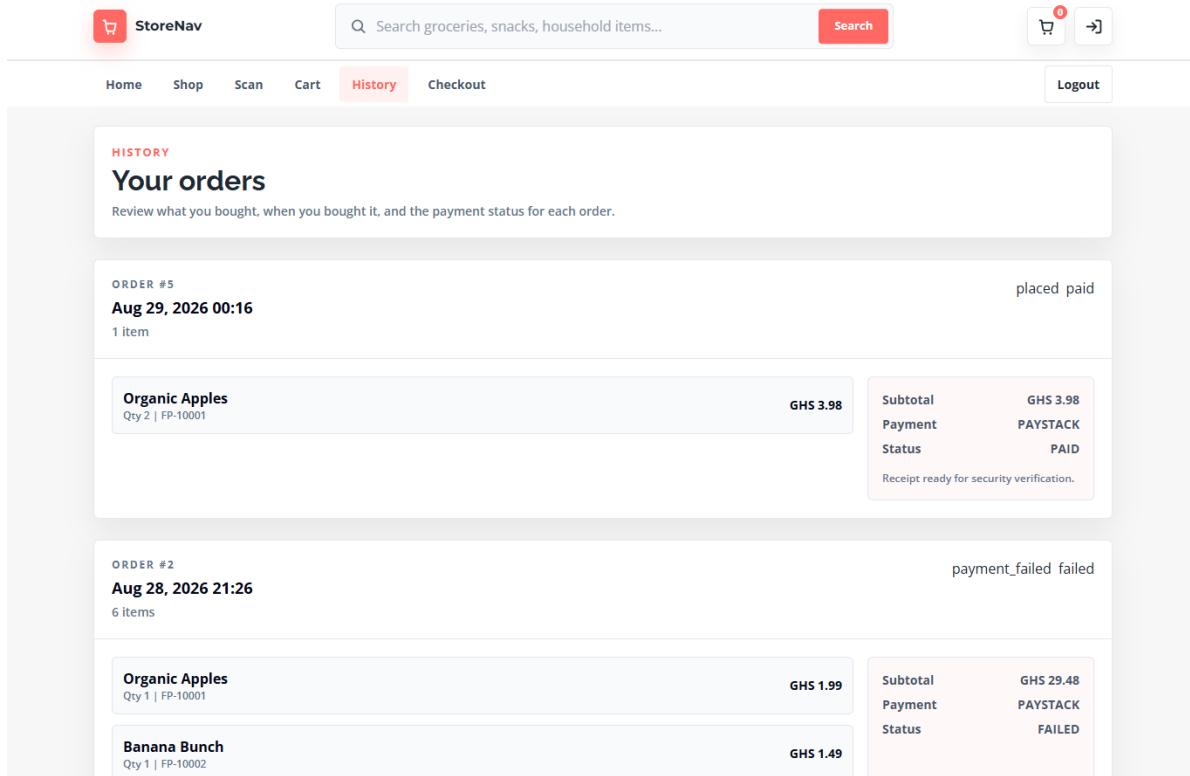


Figure 4.8: Customer history page listing previous orders and payment status.

4.1.7 Scan Page Implementation

The scan page allows customers to scan product barcodes and add matching products directly to the cart. It uses a camera-based scanner where available and sends the decoded barcode to the protected scan API. The scan result updates the cart total and helps support the self-checkout workflow.

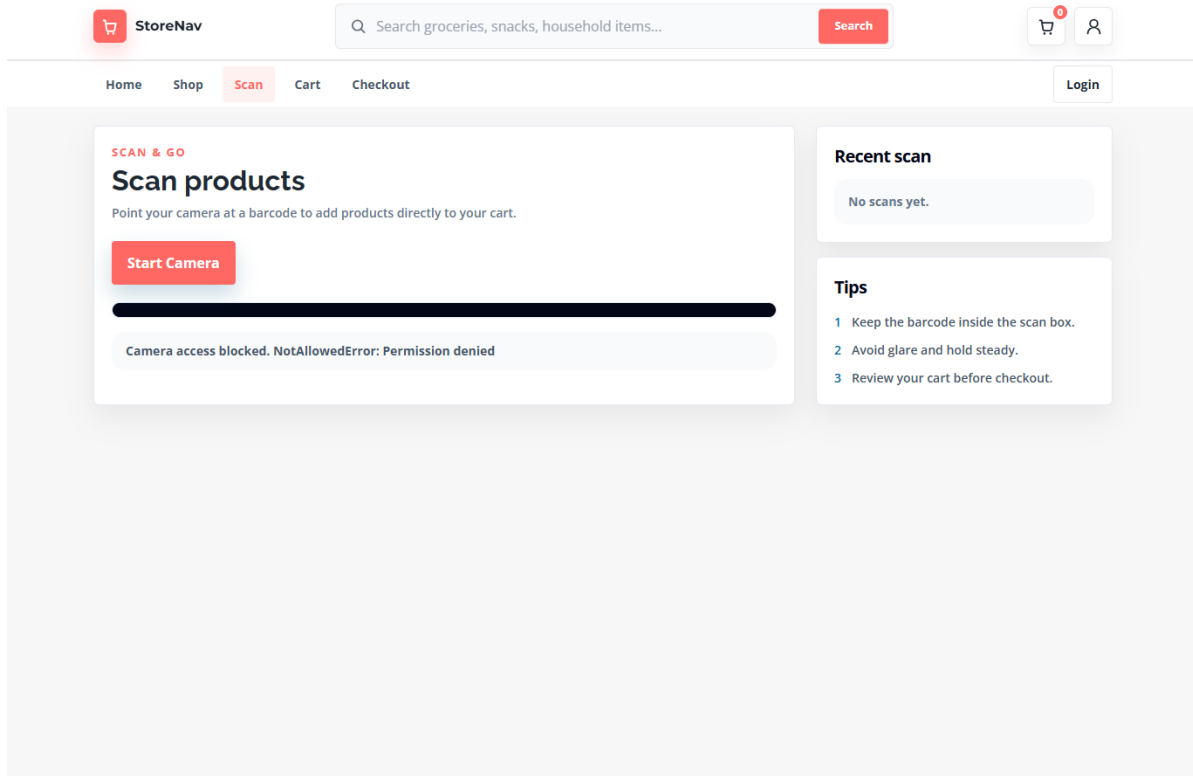


Figure 4.9: Scan page used to add products to the cart by barcode.

4.2 Authentication and Email OTP Verification

The authentication module controls user access for customers, administrators, and security staff. It supports account registration, login, logout, role-based redirection, and email OTP verification. Customers who register must verify their email address before continuing into the authenticated shopping workflow.

The login process checks credentials and then redirects users based on their role. Administrators are sent to the admin dashboard. Security staff are sent to the security portal. Customers are sent to the customer shopping area. If a customer account has not been verified, the user is logged out and routed to the OTP verification page.

4.2.1 Registration Flow

47. The customer enters name, email, password, and password confirmation.
48. The system validates the input and checks that the email is unique.
49. A customer user record is created with an unverified email status.
50. The system generates a six-digit OTP and stores a hashed version in the `email_verification_otps` table.
51. The OTP is emailed to the user through Laravel Mail.
52. The user is redirected to the OTP verification page.

The screenshot shows the StoreNav registration page. At the top, there is a navigation bar with the StoreNav logo, a search bar with the placeholder text "Search groceries, snacks, household items...", and a "Search" button. To the right of the search bar are icons for a shopping cart and a user profile. Below the navigation bar is a horizontal menu with links for "Home", "Shop", "Scan", "Cart", and "Checkout". On the far right of this menu is a "Login" button. The main content area is divided into two columns. The left column has a pink header "JOIN STORENAV" and a large heading "A smarter way to shop in-store." Below this heading is a subheading "Create an account to save carts, track budgets, scan items, and pay from your phone." There are three buttons in this column: "Barcode scan to cart", "Scan and pay in store", and "Fast checkout simulation". The right column has a pink header "ACCOUNT" and a heading "Create account". It contains four input fields: "Name", "Email", "Password", and "Confirm Password". Below these fields is a red "Create Account" button. At the bottom of the right column is a link "Already have an account? Login".

Figure 4.10: Registration page for customer account creation.

4.2.2 OTP Verification Flow

53. The user enters the six-digit OTP received by email.
54. The system checks the latest unused OTP for the pending user.
55. The system confirms that the OTP has not expired.
56. If the code is valid, the OTP is marked as used.
57. The user's email_verified_at field is updated.
58. The user is logged in and redirected to the intended page.

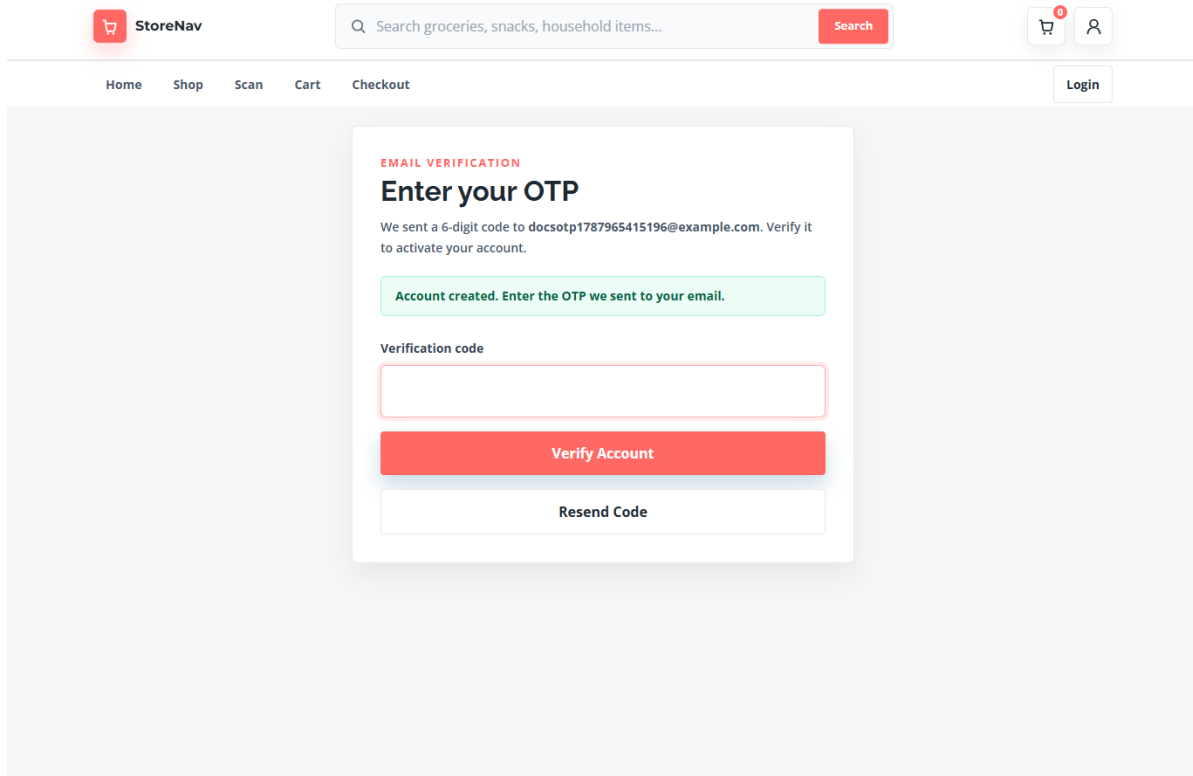


Figure 4.11: OTP verification page used after customer account registration.

4.2.3 Logout and Session Handling

Logout is available for all three roles. The logout route invalidates the session and regenerates the CSRF token. This prevents users from continuing to access authenticated routes after logging out. The logout button is visible in the admin panel, customer navigation, and security portal layout.

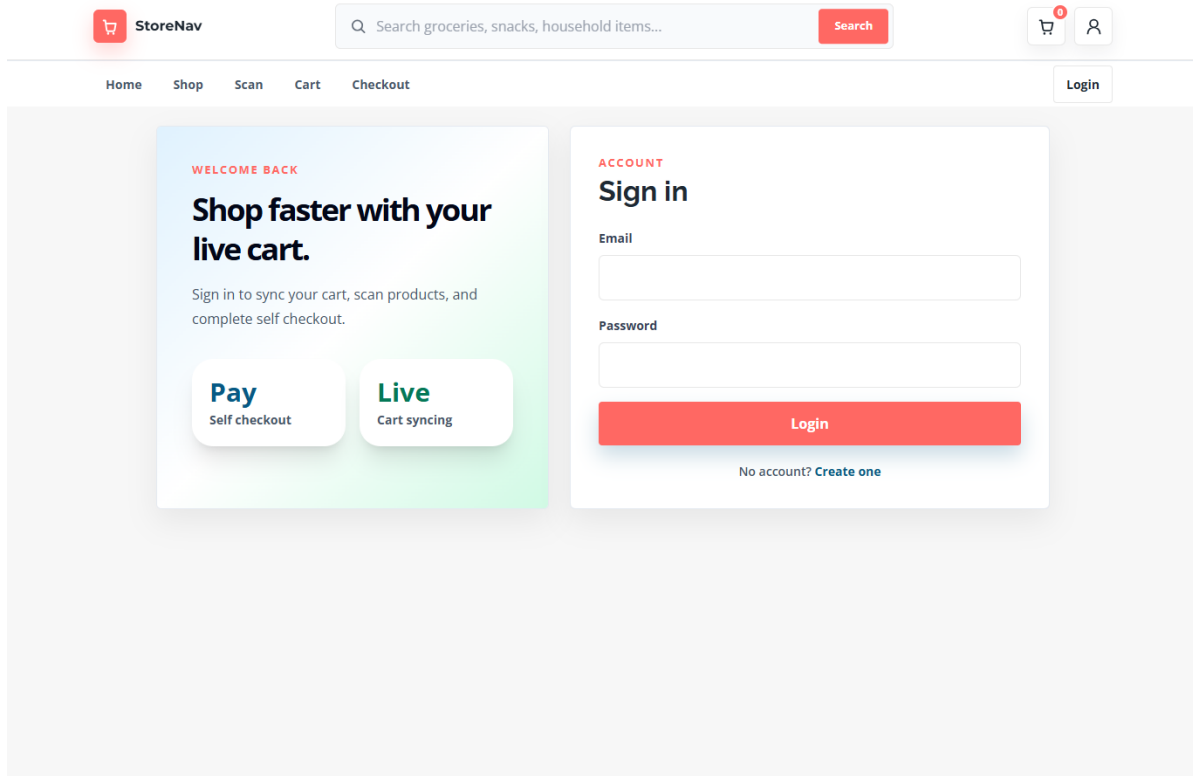


Figure 4.12: Login page and navigation area showing account access controls.

4.2.4 Unauthorized Access Handling

Protected actions route unauthenticated or expired sessions to the login page. This was implemented to avoid generic unauthorized messages. For example, if a guest tries to add products to the cart or access protected workflows, the system routes the user to login and preserves the intended redirect where practical.

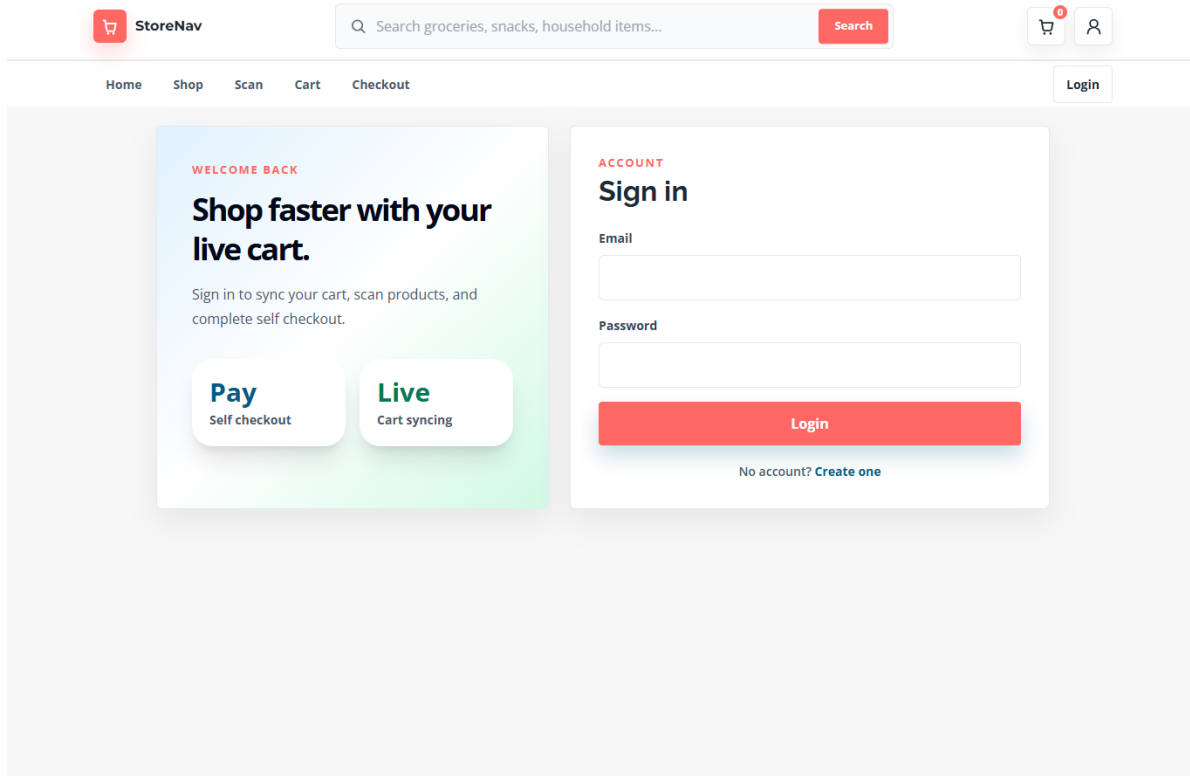


Figure 4.13: Login screen displayed when users need to authenticate before protected actions.

4.3 Admin Module

The admin module provides the operational control center for StoreNav. It uses a separate admin layout with a dark sidebar, red accent color, dashboard cards, tables, charts, and operational shortcuts. The admin interface follows the template theme requested for the system and provides a more structured back-office experience.

The admin panel is protected by authentication and role middleware. Only users with the administrator role can access the admin routes. This protects product management, order data, customer data, inventory records, analytics, and settings from ordinary customers.

4.3.1 Admin Dashboard

The dashboard presents a summary of store performance. It displays total revenue in GHS, total orders, active users, conversion rate, sales trends, top products, recent orders, and quick insights. This gives administrators a fast operational view of the store without opening separate reports.

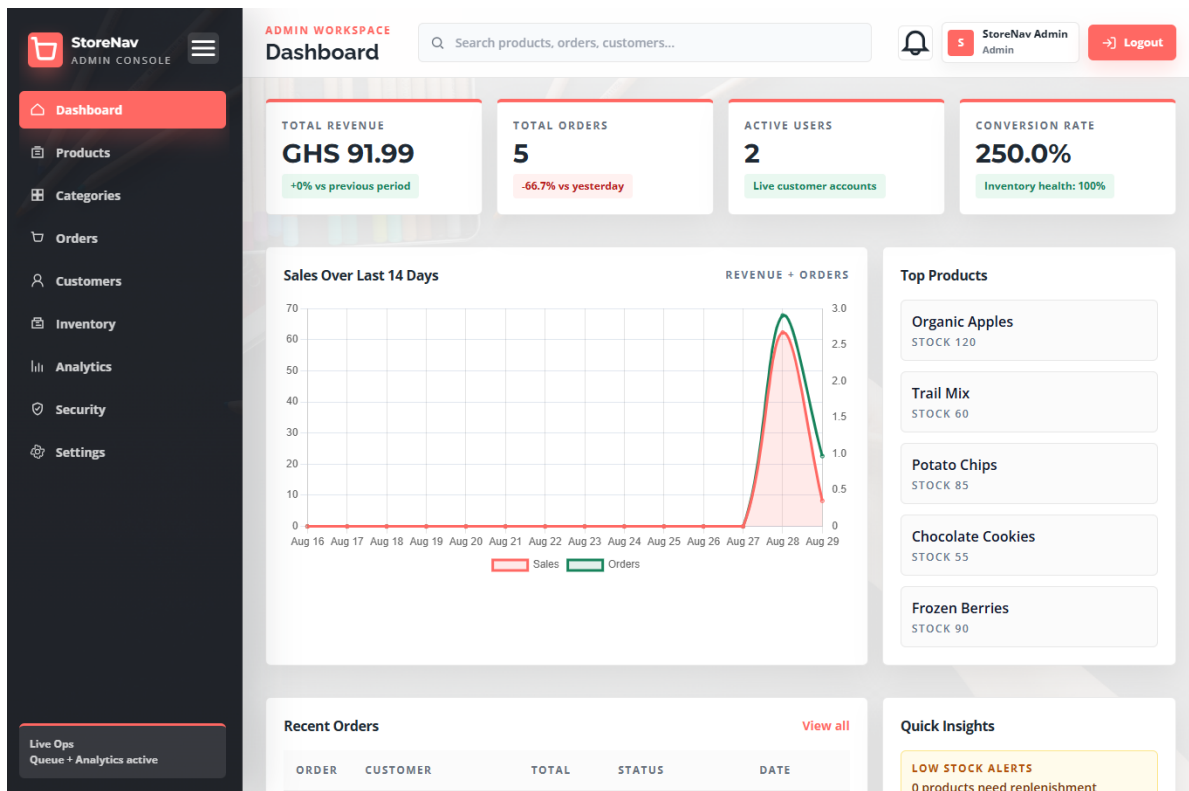


Figure 4.14: Admin dashboard showing sales, customers, orders, active users, charts, and recent orders.

4.3.2 Product Management

Product management allows the administrator to create, edit, delete, and view products. Each product can have a name, description, image, price, barcode, category, stock quantity, and active status. Product image upload is handled through the public storage disk. Product deletion also removes the stored image where applicable.

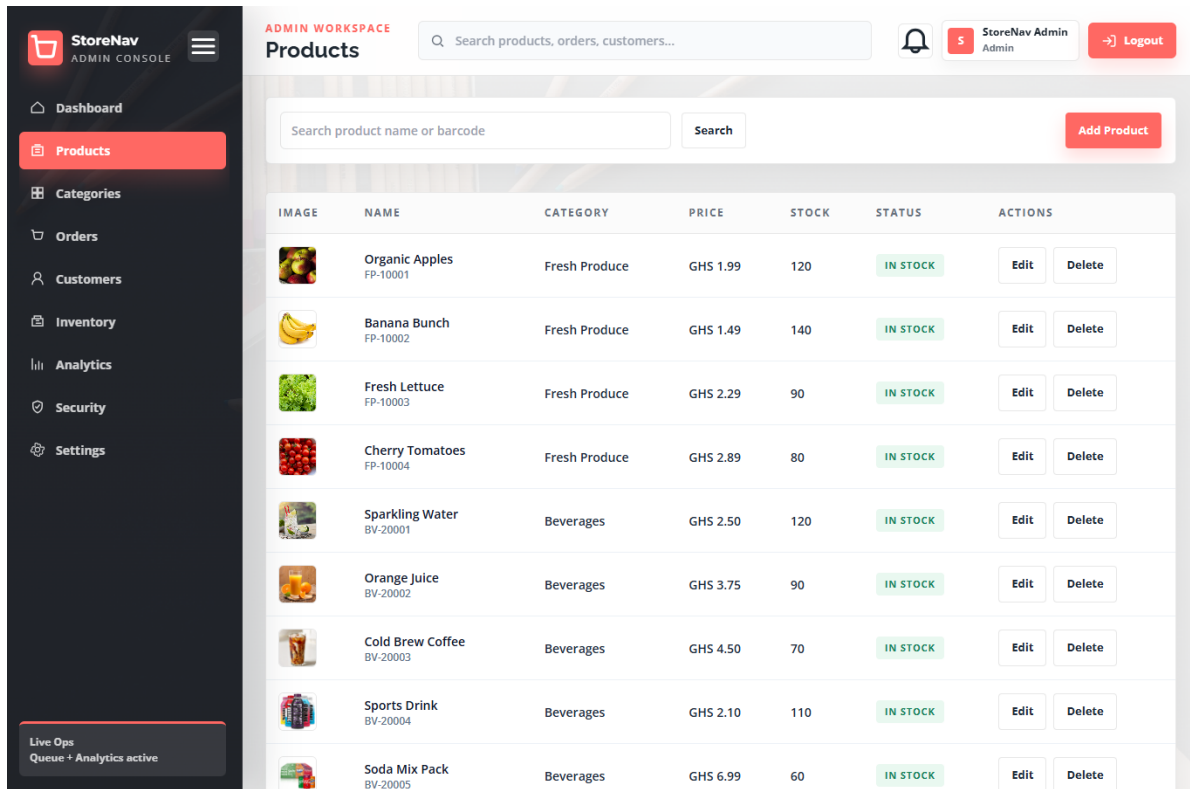


Figure 4.15: Admin product management screen with images, stock, status, and actions.

4.3.3 Category Management

Category management supports product grouping. Categories are used by the customer shop filter and by product cards to help users understand where products belong. Categories seeded into the system include Fresh Produce, Beverages, Snacks, Household, and Frozen.

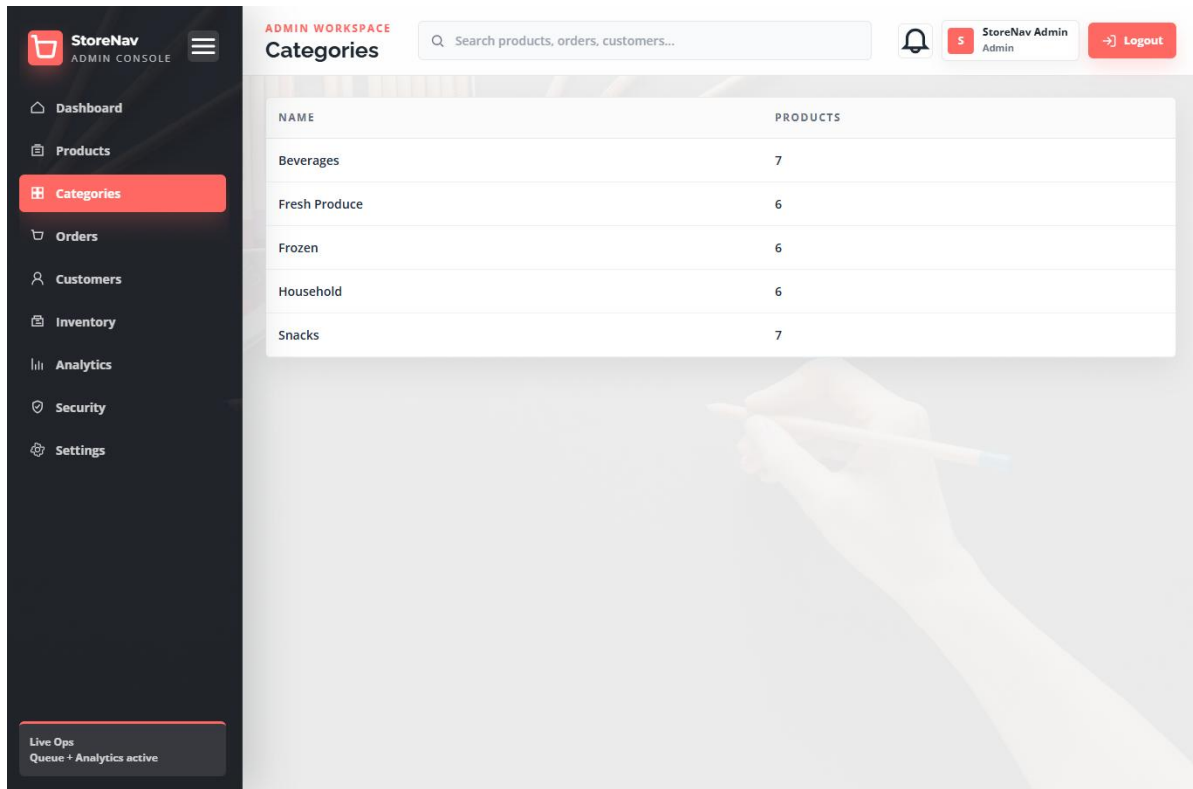


Figure 4.16: Admin category management screen showing seeded product categories.

4.3.4 Order Management

Order management gives the administrator access to purchase records. The admin can see customer names, totals, order status, payment status, and dates. This is important for business monitoring, dispute review, and transaction traceability.

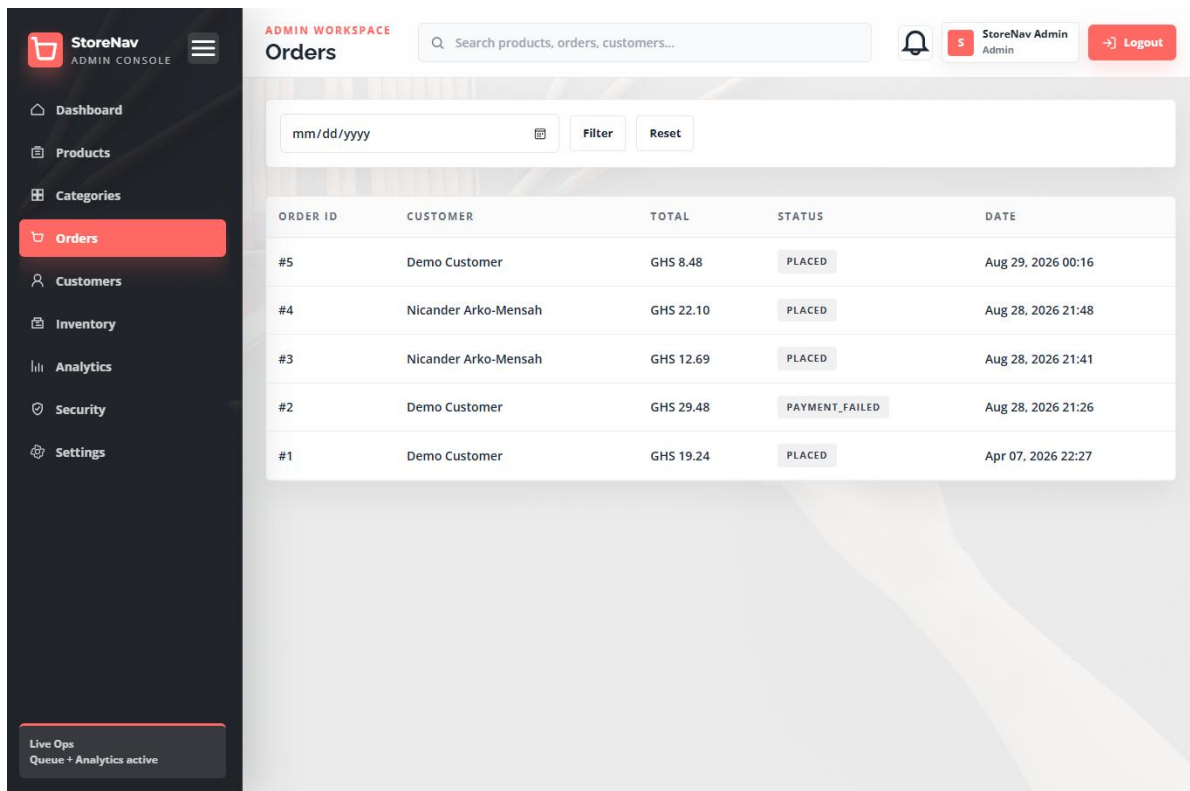


Figure 4.17: Admin order listing with customer, total, status, payment, and date filters.

4.3.5 Customer Management

Customer management lists registered customers and their order totals. This helps the administrator understand customer activity and identify active users. It also supports future expansion such as customer support, loyalty tracking, and account review.

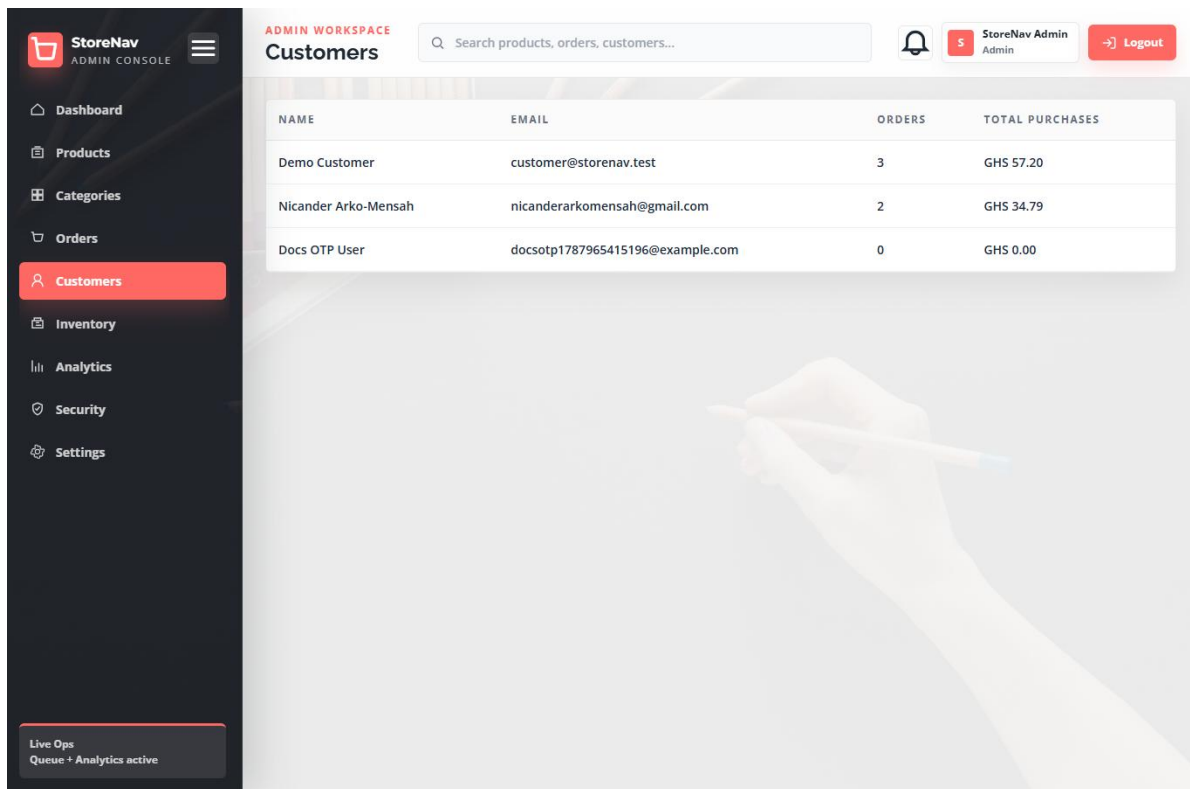


Figure 4.18: Admin customer management screen showing customer order statistics.

4.3.6 Inventory and Analytics

Inventory management is supported through product stock quantities and inventory logs. When orders are fulfilled, stock can be reduced and inventory movement can be recorded. Analytics data supports monitoring of product views, shopping behavior, and operational trends.

PRODUCT	CATEGORY	STOCK	STATUS
Laundry Detergent	Household	40	HEALTHY
Fish Fillets	Frozen	45	HEALTHY
Glass Cleaner	Household	50	HEALTHY
Chocolate Cookies	Snacks	55	HEALTHY
Paper Towels	Household	55	HEALTHY
Soda Mix Pack	Beverages	60	HEALTHY
Trail Mix	Snacks	60	HEALTHY
Pretzel Twists	Snacks	65	HEALTHY
Ice Cream Tub	Frozen	65	HEALTHY
Cold Brew Coffee	Beverages	70	HEALTHY
Frozen Pizza	Frozen	70	HEALTHY
Granola Bars	Snacks	75	HEALTHY
Cherry Tomatoes	Fresh Produce	80	HEALTHY

Figure 4.19: Admin inventory page showing product stock levels and availability.

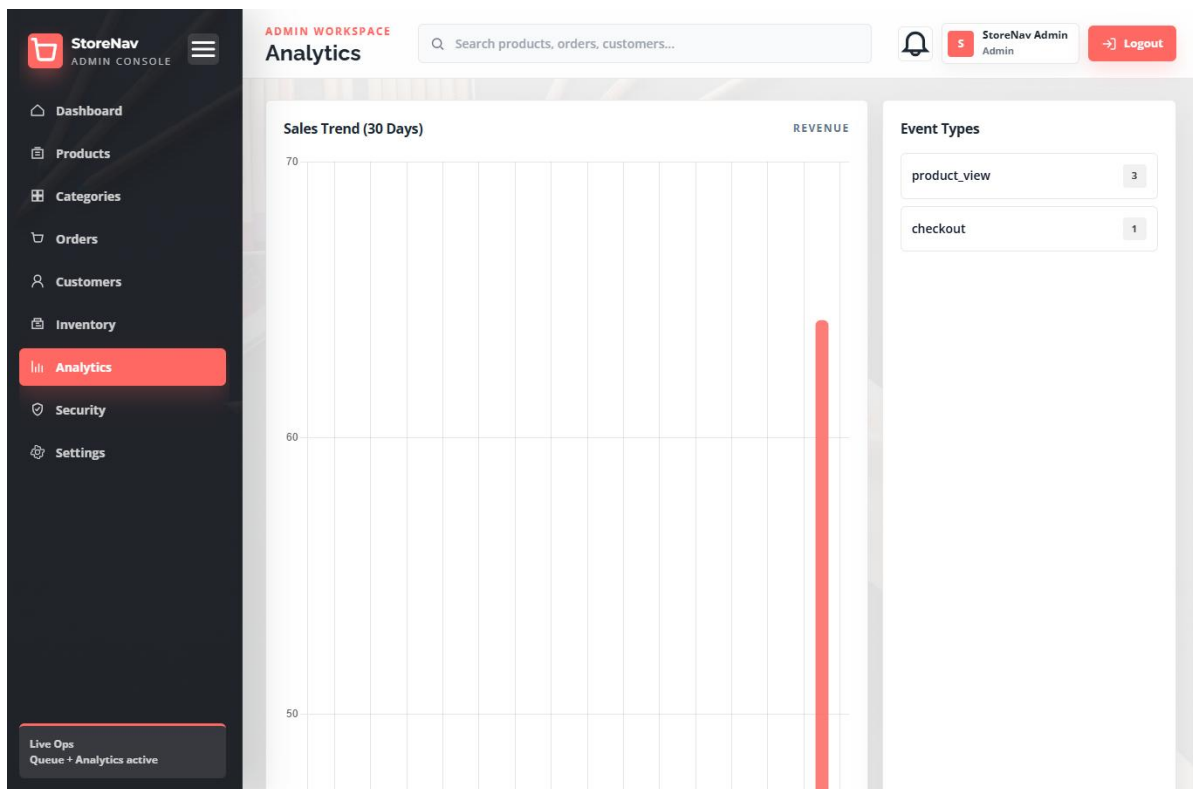


Figure 4.20: Admin analytics page showing product-view activity and popular items.

4.3.7 Settings Page

The settings page provides a system summary for administrators. It shows product, customer, order, and category counts and lists operational preferences such as currency, image upload behavior, and receipt support.

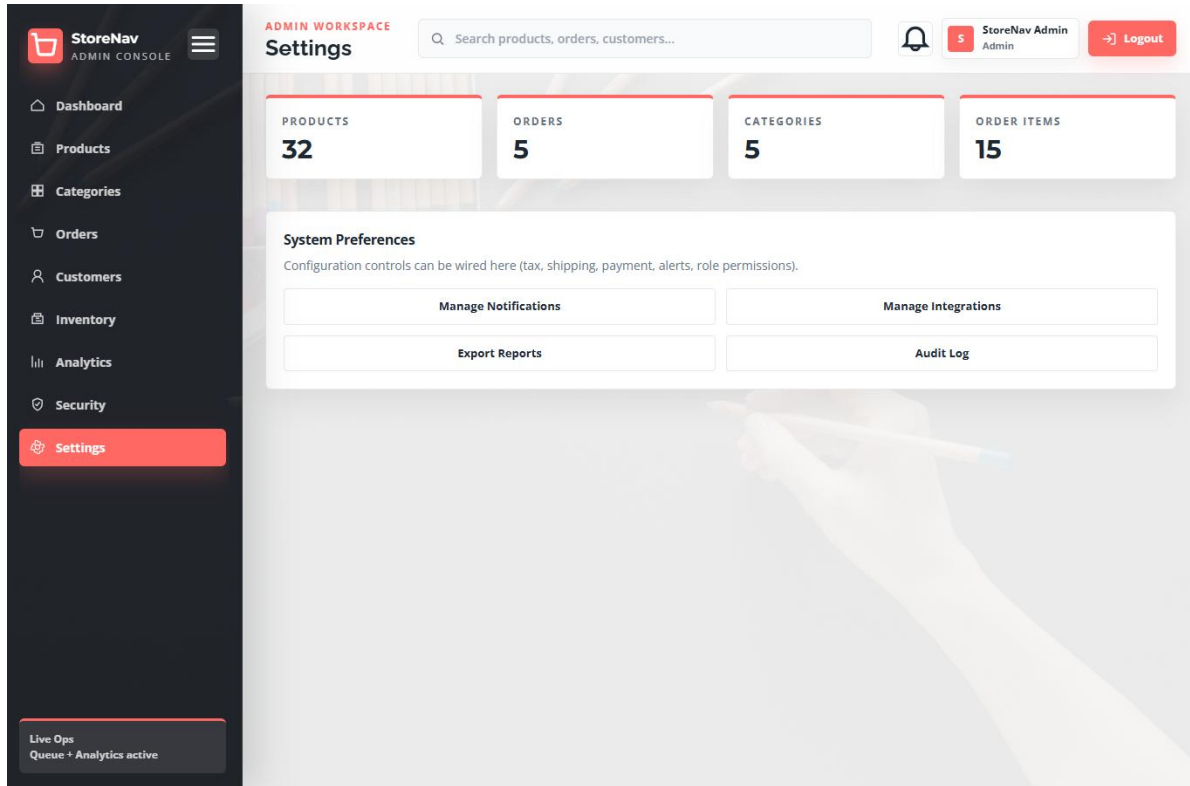


Figure 4.21: Admin settings page showing system counts and preferences.

4.4 Security Portal

The security portal is designed for supermarket exit verification. It is used after the customer has paid and received an email receipt. The portal allows security staff to scan the receipt QR code or enter the receipt token manually. The system then displays the matching order for verification.

The portal prevents a customer from simply showing an untrusted image or unrelated code. The QR code points to the system's own receipt verification route. When scanned, the route loads the order using the receipt token stored in the database. If the token is invalid, the system does not display a valid order.

4.4.1 QR Scanning

The QR scanner uses a browser-based scanner interface. Security staff start the scanner, point the camera at the customer's receipt QR code, and the system reads the encoded verification URL. If the QR code contains a valid StoreNav receipt route, the portal redirects to the receipt details page.

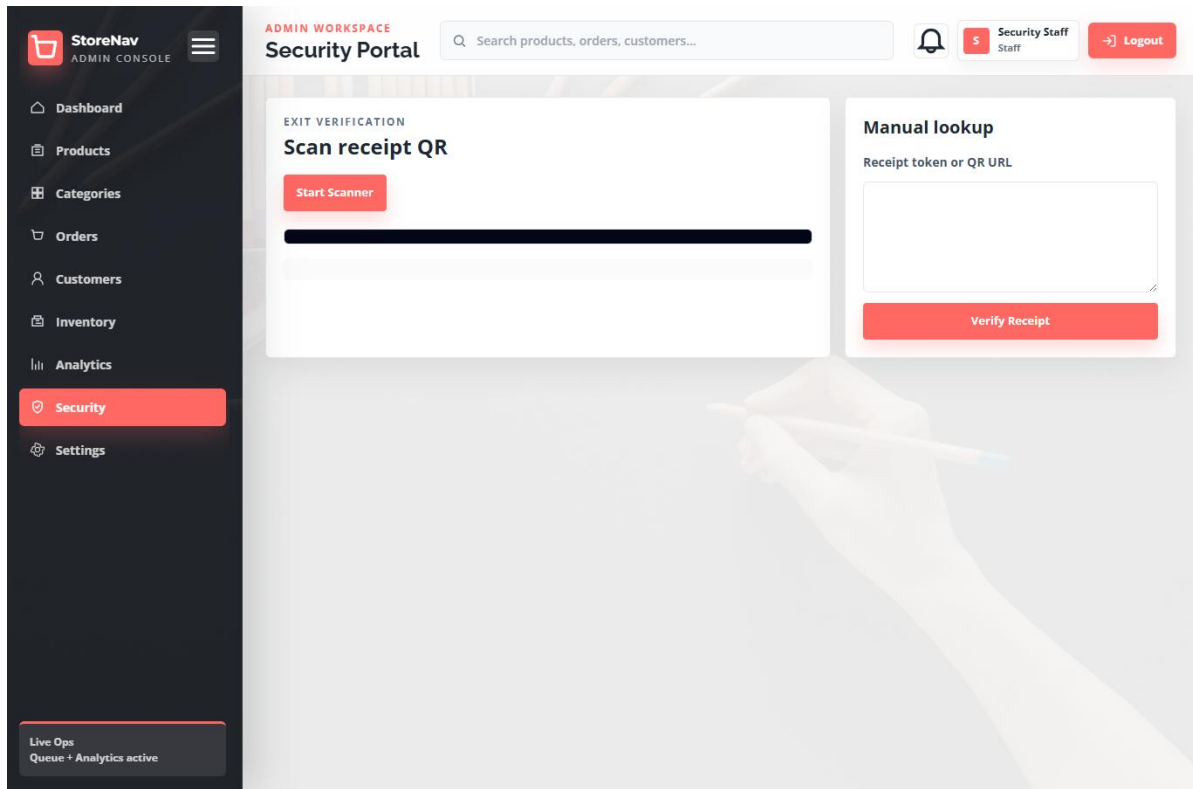


Figure 4.22: Security portal scanner used to read receipt QR codes.

4.4.2 Manual Lookup

Manual lookup is provided as a fallback for situations where the camera is unavailable, the QR code cannot be scanned, or the staff member needs to enter a receipt token directly. This improves reliability because verification does not depend only on camera scanning.

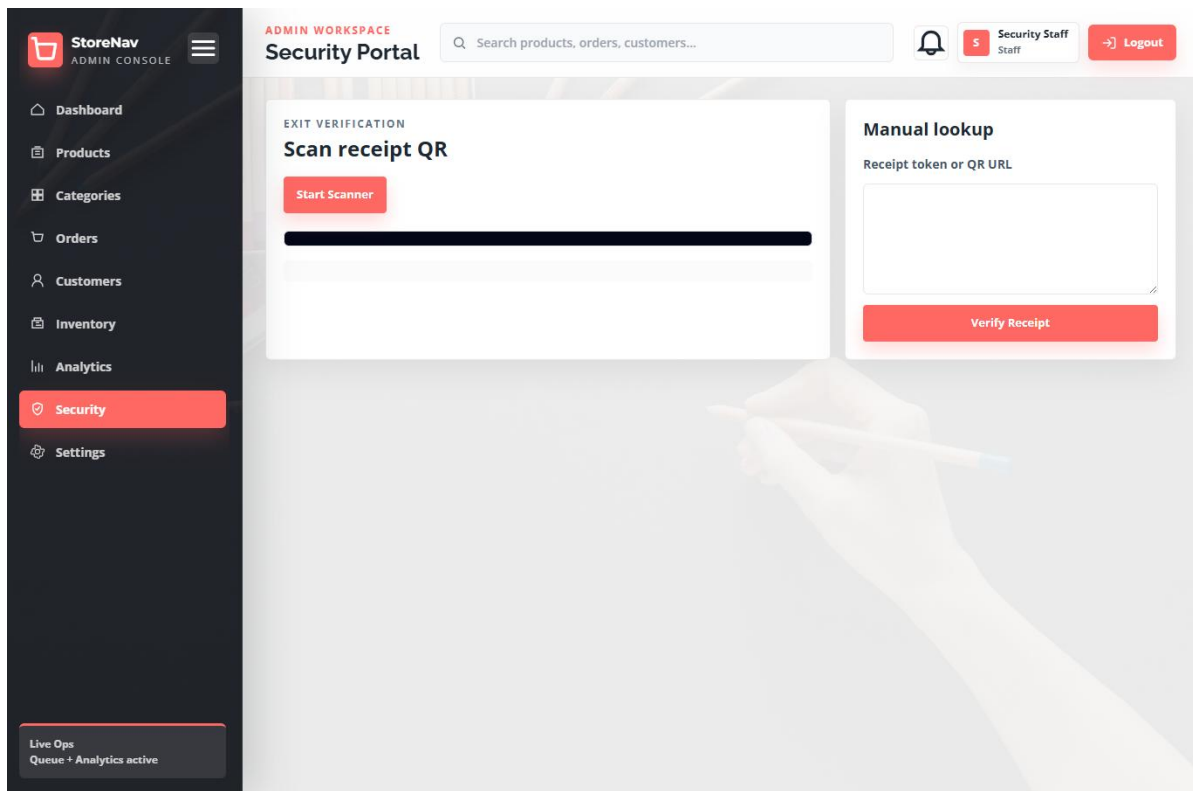


Figure 4.23: Manual receipt lookup field available inside the security portal.

4.4.3 Verification Record

When a valid receipt is verified, the order record is updated with `receipt_verified_at` and `receipt_verified_by`. This records the exact time of verification and the staff account that performed it. If a receipt has already been verified, the portal can display the previous verification status to reduce repeated verification confusion.

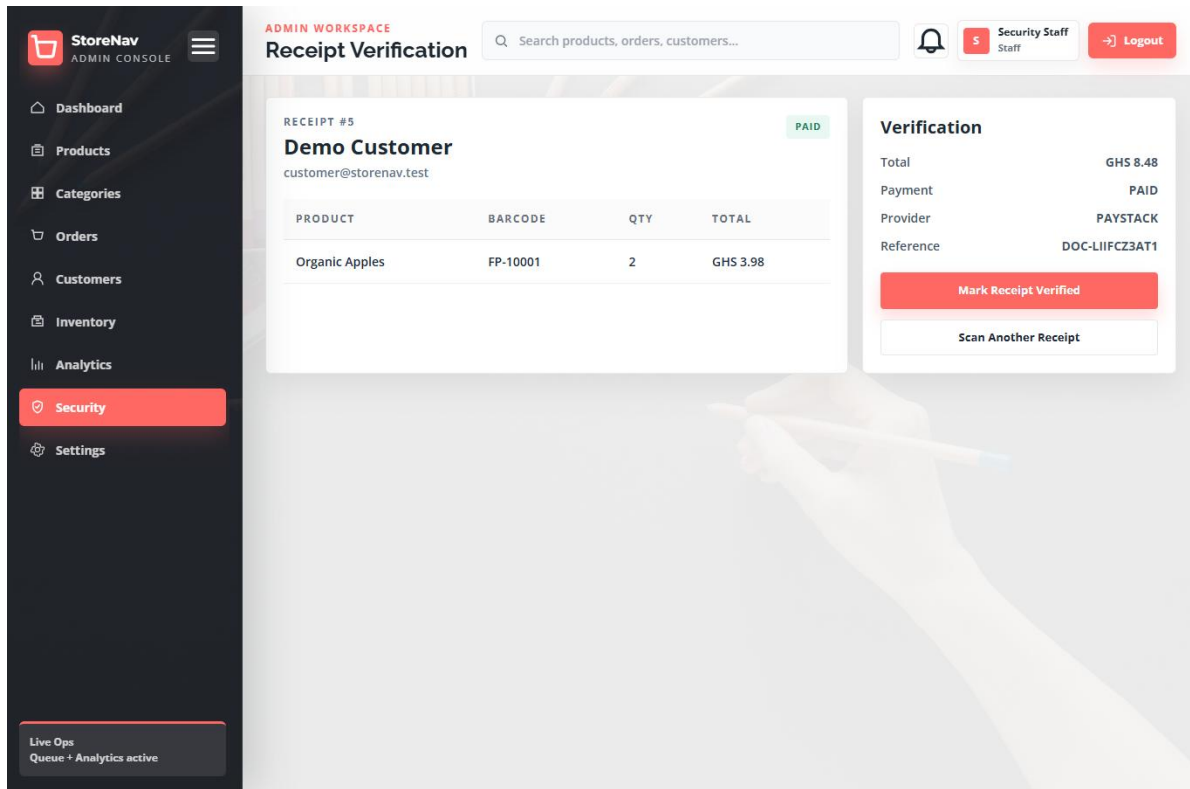


Figure 4.24: Receipt verification detail screen showing order items and verification controls.

4.5 Payment and Receipt Workflow

Payment and receipt verification form the core transaction workflow of StoreNav. The workflow ensures that a customer cannot receive a valid receipt until payment has been verified. The receipt then becomes the proof of purchase used by security staff.

59. The customer scans or adds products to the active cart.
60. The checkout API creates an order and payment record.
61. Paystack initializes the transaction using the order total in GHS.
62. The customer completes payment on Paystack.
63. The Paystack callback verifies the reference and amount.
64. The order is marked paid and fulfilled.
65. The system emails a receipt with the purchased items and QR verification code.
66. Security scans the QR code and verifies the receipt before the customer exits.

4.5.1 Paystack Initialization

PaystackService receives the order and callback URL. It sends the customer's email, order amount, currency, and callback URL to Paystack. The amount is converted to the smallest currency unit expected by Paystack. If Paystack responds successfully, the authorization URL is returned to the frontend for payment redirection.

4.5.2 Paystack Callback Verification

After payment, Paystack redirects the browser to the application callback route with a reference. The `PaystackPaymentController` finds the matching payment record, calls `PaystackService` to verify the reference, checks that the transaction is successful, confirms that the amount matches the order, and then updates the payment and order records.

4.5.3 Order Fulfillment

`OrderFulfillmentService` marks the order as placed and paid, sets fulfillment timestamps, generates a receipt token if one does not exist, clears the checked-out cart, dispatches order processing, and sends the receipt email. This keeps fulfillment behavior centralized instead of scattering it across controllers.

4.5.4 Email Receipt and QR Code

`OrderReceiptMail` builds the email receipt after payment. The receipt displays purchased products, item totals, and the final total in GHS. The QR code is generated from the security receipt verification URL and embedded directly into the email body so that mail clients can render it more reliably than a detached or data URI-only image.

4.6 Database Design

The database design supports a complete self-checkout workflow. It separates account data, product data, cart data, order data, payment data, inventory data, and verification data. This separation improves maintainability and makes it easier to audit transactions.

Table	Purpose
users	Stores customers, administrators, and security staff with role and email verification fields.
categories	Stores product categories used for filtering and grouping.
products	Stores product name, description, image path, price, barcode, category, stock quantity, and active status.
carts	Stores active and checked-out carts with GHS totals.
cart_items	Stores products and quantities selected before checkout.
orders	Stores placed orders, payment status, receipt token, receipt sent date, verification date, and verifier.
order_items	Stores final purchased product lines for each order.
payments	Stores Paystack provider, amount, status, reference, payload, and paid date.
inventory_logs	Stores stock movement after checkout.
email_verification_otps	Stores OTP codes, expiry dates, and used timestamps.

4.6.1 Main Entity Relationships

- A user can have many carts and many orders.
- A category can have many products.
- A product belongs to one category and can appear in many cart items and order items.
- A cart belongs to a user and contains many cart items.

- An order belongs to a user and can belong to a cart.
- An order contains many order items and has one payment record.
- A payment belongs to an order and stores provider-specific transaction details.
- A receipt verification record is stored on the order through receipt token, verification timestamp, and verifier fields.
- An email OTP belongs to a user and is consumed after successful verification.

4.6.2 Important Database Fields

Entity	Important Fields	Reason
users	role, email_verified_at, budget_limit	Controls access, verification status, and customer spending support.
products	barcode, image_path, price, stock_quantity, is_active	Supports scanning, product display, pricing, inventory, and catalog visibility.
carts	status, subtotal, tax, total, currency	Tracks active and checked-out cart totals in GHS.
orders	status, payment_status, receipt_token, receipt_sent_at, receipt_verified_at	Tracks purchase, payment, receipt delivery, and security verification.
payments	provider, amount, status, reference, payload, paid_at	Stores Paystack transaction details and verification evidence.
email_verification_otps	code, expires_at, used_at	Supports secure account verification and prevents OTP reuse.

4.6.3 Seeder Design

The database seeder is designed to be idempotent and hosting-safe. It can create or update the admin user, security user, optional demo customer, categories, and products without duplicating records on repeated runs. Production credentials are read from environment variables so live hosting passwords are not hardcoded in the source file.

4.7 Key Source Modules

Component	File or Class	Responsibility
Customer pages	CustomerController	Loads home, shop, scan, cart, checkout, product, and history views.
Cart logic	CartService	Creates active carts, adds/removes products, recalculates totals, and builds API payloads.
Checkout	Api CheckoutController	Creates orders and initializes Paystack or handles supported checkout providers.
Payment callback	PaystackPaymentController	Verifies payment references and triggers order fulfillment.
Fulfillment	OrderFulfillmentService	Marks orders paid, clears carts, dispatches stock processing, and sends receipts.
Receipt email	OrderReceiptMail	Builds the receipt email and embeds the QR code image.
Security verification	SecurityPortalController	Looks up receipt tokens, displays order details, and marks receipts verified.

Access control	RoleMiddleware	Protects role-specific routes such as admin pages.
Seeder	DatabaseSeeder	Seeds admin, security, optional customer, categories, and products for deployment.

4.8 Route Design

The route design separates public customer pages, authentication routes, protected customer pages, security routes, admin routes, and API routes. This organization improves readability and makes it easier to apply middleware to specific groups.

Route Group	Examples	Purpose
Public customer routes	/, /shop, /scan, /cart, /checkout, /products/{product}	Displays customer shopping screens and product details.
Authentication routes	/login, /register, /verify-email-otp, /logout	Handles account access, OTP verification, and logout.
Security routes	/security, /security/lookup, /security/receipts/{token}	Allows staff to scan, look up, display, and verify receipts.
Admin routes	/admin, /admin/products, /admin/orders, /admin/inventory	Allows administrators to manage store operations.
API routes	/api/products, /api/cart/add, /api/scan, /api/cart, /api/checkout	Supports JavaScript-driven product, cart, scan, and checkout operations.

4.9 Interface Design Decisions

The user interface was redesigned to fit an online shop and supermarket self-checkout workflow. The customer interface uses product cards, compact categories, scan actions, cart counters, GHS prices, and a clear navigation structure. The admin interface uses a darker operational dashboard theme with red accents to match the selected template direction.

- The virtual room was removed from the active system because the final direction is self-checkout, not room navigation.
- Product cards were reduced in size to prevent the shop from looking stretched or oversized.
- Ratings were removed from product cards to reduce visual clutter.
- Product name and category spacing was tightened to create a more compact product grid.
- The checkout page retains navigation so customers can return to the cart or shop.
- SweetAlert replaces generic browser alerts and confirmations.
- The login link appears on the right side of the desktop navbar.
- Logout access is available for customer, admin, and security roles.

4.10 Security Design

Security design in StoreNav is based on authentication, role separation, email verification, CSRF protection, protected API routes, payment verification, and receipt token validation. These controls reduce unauthorized access and help protect transaction integrity.

Security Control	Implementation	Purpose
------------------	----------------	---------

Authentication	Laravel session authentication.	Confirms user identity before protected actions.
Role control	RoleMiddleware and route groups.	Separates customer, admin, and security permissions.
Email verification	Six-digit OTP and email_verified_at field.	Confirms customer ownership of registered email.
Payment validation	Paystack reference and amount verification.	Prevents unpaid orders from being fulfilled.
Receipt token	Unique token stored on order.	Links the email QR code to a specific paid order.
CSRF protection	Laravel form and request tokens.	Protects session-based form submissions.

4.11 Hosting and Deployment Design

The system includes Hostinger deployment preparation. A Hostinger-specific environment file template was created with production settings for MySQL, secure sessions, public file storage, synchronous queue execution, Paystack configuration, and SMTP mail. A deployment guide explains where to upload the Laravel project and how to point the document root to the public directory.

- APP_ENV should be production and APP_DEBUG should be false.
- APP_URL should point to the real HTTPS domain.
- DB_CONNECTION should use MySQL on Hostinger.
- FILESYSTEM_DISK should be public for uploaded product images.
- QUEUE_CONNECTION should be sync on shared hosting unless a worker is available.
- Production admin and security passwords should be supplied through environment variables before seeding.

4.12 Chapter Summary

This chapter presented the detailed design and implementation of StoreNav. It explained the layered architecture, customer module, authentication and OTP flow, admin module, security portal, Paystack payment workflow, email receipt and QR code design, database structure, source modules, route organization, interface decisions, security controls, and deployment preparation. The next chapter presents testing, deployment, conclusion, and recommendations.

CHAPTER FIVE

TESTING, DEPLOYMENT, CONCLUSION, AND RECOMMENDATIONS

5.0 Testing Summary

The system includes automated tests for cart operations, checkout, API cart access, API checkout, receipt verification, customer history, and email OTP verification. The latest test run passed all available tests, confirming the stability of the main functional paths.

Test Area	Purpose
Cart service	Confirms duplicate product scans increase quantity correctly.
Checkout	Confirms checkout creates an order from the active cart.
API cart	Confirms products can be added through authenticated API calls.
Receipt verification	Confirms security can verify a paid receipt.
Customer history	Confirms customers can view their order history.
Email OTP	Confirms registration requires OTP verification and accepts valid codes.

5.1 Deployment Preparation

The project has been prepared for Hostinger hosting using a production environment template and deployment guide. The recommended setup is to upload the Laravel project outside `public_html` and point the domain document root to the public directory. If the plan does not support custom document roots, the contents of the public directory can be moved into `public_html` and `index.php` adjusted to point back to the project root.

- Set `APP_ENV` to production and `APP_DEBUG` to false.
- Use MySQL credentials from Hostinger for `DB_DATABASE`, `DB_USERNAME`, and `DB_PASSWORD`.
- Set `APP_URL` to the real HTTPS domain.
- Set `QUEUE_CONNECTION` to sync for shared hosting unless a worker is configured.
- Set `FILESYSTEM_DISK` to public and run `php artisan storage:link`.
- Run migrations and the production-safe database seeder.
- Provide real Paystack and mail credentials before accepting live transactions.

5.2 Security Considerations

- Only verified accounts should continue into protected customer workflows.
- Admin routes are protected by authentication and the admin role middleware.
- Security routes require authentication and allow only staff or admin users.
- Receipt tokens should be unique and difficult to guess.
- Paystack payment status and amount should be verified before fulfillment.
- Production credentials must be stored in the server `.env` file and not exposed publicly.

5.3 Conclusion

StoreNav has been redesigned from a navigation-oriented prototype into a practical online smart shopping and self-checkout system. The implemented system supports product browsing, barcode scanning, cart management, GHS pricing, Paystack payment, email OTP verification, email receipts, QR receipt verification, order history, role-based administration, and Hostinger deployment preparation.

5.4 Recommendations

- Add live supermarket product images and maintain product records regularly.
- Use Paystack live keys only after full payment testing with real settlement accounts.
- Configure a reliable SMTP provider before hosting to support OTP and receipt delivery.
- Consider adding staff audit reports for verified receipts.
- Extend analytics to show product demand, abandoned carts, and peak shopping periods.
- Review production security headers, backups, and database permissions before launch.

REFERENCES

Laravel project source code for StoreNav, controllers, services, routes, migrations, and Blade templates.

Paystack service integration configuration and payment callback implementation in the StoreNav source code.

StoreNav Hostinger deployment guide and production environment template prepared with the project.

Captured screenshots from the running StoreNav application on the local development server.

APPENDIX A: HOSTING CHECKLIST

- Upload project files to Hostinger.
- Create and configure MySQL database.
- Copy HOSTINGER.env.example values into the production .env file.
- Generate APP_KEY if it has not already been generated.
- Run `php artisan migrate --force`.
- Run `php artisan db:seed --class=DatabaseSeeder --force`.
- Run `php artisan storage:link`.
- Run `php artisan config:cache` and `php artisan route:cache`.
- Confirm login, registration OTP, product browsing, Paystack checkout, receipt email, and security verification.